



**COMSATS University, Islamabad Pakistan**

## **Seevia**

**AI-Powered Assistive Ecosystem for Visually Impaired Users**

***By***

**Malaika Junaid      CIIT/FA22-BAI-020/ISB**

**Syeda Aleeza Tahir      CIIT/FA22-BAI-038/ISB**

***Supervisor***

**Dr. Samera Batool**

A handwritten signature in blue ink, appearing to be "S. Batool", with a small "v." at the end.

***Bachelor of Science in Artificial Intelligence (2022-2026)***

**The candidate confirms that the work submitted is their own, and appropriate credit has been given where reference has been made to the work of others.**



**COMSATS University, Islamabad Pakistan**

# **Seevia**

.

**A project presented to  
COMSATS University, Islamabad**

**In partial fulfillment  
of the requirement for the degree of**

***Bachelor of Science in Artificial Intelligence (2022-2026)***

**By**

**Malaika Junaid  
Syeda Aleeza Tahir**

**CIIT/FA22-BAI-020/ISB  
CIIT/FA22-BAI-038/ISB**

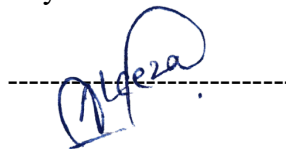
# DECLARATION

We hereby declare that this software, neither whole nor as a part, has been copied out from any source. It is further declared that we have developed this software and accompanied the report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be a reproduction of some other, we will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Malaika Junaid

A handwritten signature in blue ink, appearing to read 'Malaika Junaid', is written over a horizontal dashed line.

Syeda Aleeza Tahir

A handwritten signature in blue ink, appearing to read 'Syeda Aleeza Tahir', is written over a horizontal dashed line.

# CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (AI) Seevia was developed by **MALAIKA JUNAID (CIIT/FA22-BAI-020)** and **SYEDA ALEEZA TAHIR (CIIT/FA22-BAI-038)** under the supervision of DR. SAMERA BATOOL and that in their opinion; it is fully adequate, in scope and quality for the degree of Bachelor of Science in Artificial Intelligence.



Samera Batool

**Supervisor**

-----  
**External Examiner**

-----  
**Head of Department**  
**(Department of Computer Science)**

# EXECUTIVE SUMMARY

Seevia is a voice-first, multimodal AI assistive mobile application designed to empower visually impaired individuals (PWDs) in Pakistan to perform everyday tasks independently. Pakistan has over 1.8 million people living with some form of visual impairment, many of whom lack access to effective, localized assistive technology. Existing solutions such as Seeing AI, Be My Eyes, and Aira are either English-only, internet-dependent, human-reliant, or prohibitively expensive, leaving a significant accessibility gap for Pakistani users. Seevia bridges this gap through an integrated, AI-powered ecosystem that requires only a mid-range Android smartphone.

The system delivers six tightly integrated modules: (1) a Voice Command Interface supporting Urdu and English mixed-language input via Deepgram STT with 7-class NLP intent recognition; (2) AI Personalization using Markov-chain behavioral analysis to predict and surface proactive suggestions; (3) a Pantry Management module powered by YOLOv8-Nano object detection and Google ML Kit OCR for product identification, expiry tracking, and shopping list generation; (4) a Shopping Assistant combining camera-based product recognition with reinforcement learning-driven store navigation; (5) an Emergency Management system using sensor fusion (accelerometer + gyroscope) for automated fall detection and multi-channel SOS escalation; and (6) a Volunteer Matching network for community-based emergency response.

Seevia is implemented in React Native with Expo, using Firebase for real-time backend services (Authentication, Firestore, Cloud Messaging, Storage). A three-tier AI fallback architecture routes inference from on-device TensorFlow Lite models to HuggingFace APIs and cloud services (Google Vision API, Gemini) based on confidence thresholds, ensuring offline-first resilience. The YOLOv8-Nano model, trained on 5,722 images across 23 Pakistani product classes, achieved mAP@50 of 87.3% and 91% real-world accuracy. All user-facing audio is delivered in Urdu/English with haptic feedback, WCAG 2.1 AA compliance, and a fully eyes-free interaction model.

## **Abstract**

Seevia is being developed to assist visually impaired individuals by providing a smart assistant that supports their independence, safety, and daily living through voice-driven, accessible technology. Currently, these individuals face significant challenges in completing everyday tasks such as shopping, navigating stores, and tracking household inventory due to limited accessible tools. While some assistive technologies exist, they often lack integration, voice control, or personalization, leaving users dependent on others for routine activities. The gap addressed by this project is the absence of an all-in-one, AI-enabled assistant that helps users independently manage household tasks, shopping, and emergencies through intuitive voice and camera-based interaction. It seeks to resolve these issues by offering a fully voice-operated solution with features like product recognition, pantry tracking, and emergency support. Seevia is designed to enable users to perform essential tasks through voice commands and camera input, with support for personalized profiles, inventory tracking, and real-time product information. The ultimate goal is to streamline these activities into a seamless experience, improving autonomy, and reducing reliance on caregivers. It will promote self-sufficiency by learning from user behavior and adapting over time to better meet individual necessities. The app is intended to increase the quality of life for visually impaired users by integrating essential functions into one accessible mobile platform. This comprehensive assistant will significantly boost user independence and safety through seamless, voice-first interaction tailored to their unique needs.

# Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor, Dr. Samera Batool. Without her personal supervision, advice, and valuable guidance, completion of this project would have been doubtful. We are grateful to her for her encouragement and continual help throughout this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us up with the values of honesty and hard work.

And lastly, we would like to express our heartfelt gratitude to each other who stuck together through every high and every low of this journey.

Malaika Junaid



Syeda Aleeza Tahir



# Abbreviations

|        |                                                 |
|--------|-------------------------------------------------|
| AI     | Artificial Intelligence                         |
| API    | Application Programming Interface               |
| CV     | Computer Vision                                 |
| DL     | Deep Learning                                   |
| DQN    | Deep Q-Network                                  |
| FCM    | Firebase Cloud Messaging                        |
| FR     | Functional Requirement                          |
| FYP    | Final Year Project                              |
| GPS    | Global Positioning System                       |
| mAP    | Mean Average Precision                          |
| ML     | Machine Learning                                |
| NFR    | Non-Functional Requirement                      |
| NLP    | Natural Language Processing                     |
| OCR    | Optical Character Recognition                   |
| PWD    | Person with Disability (Visual Impairment)      |
| RL     | Reinforcement Learning                          |
| SDK    | Software Development Kit                        |
| SOS    | Save Our Souls (Emergency Signal)"              |
| SRS    | Software Require Specification                  |
| STT    | Speech-to-Text                                  |
| TFLite | TensorFlow Lite                                 |
| TTS    | Text-to-Speech                                  |
| UI     | User Interface                                  |
| UX     | User Experience                                 |
| WCAG   | Web Content Accessibility Guidelines            |
| YOLO   | You Only Look Once (object detection framework) |

# Table of Contents

|                                                                                |    |
|--------------------------------------------------------------------------------|----|
| Abstract .....                                                                 | v  |
| 1 Chapter 1: Introduction and Problem Definition .....                         | 1  |
| 1.1 Overview of the Project .....                                              | 1  |
| 1.2 Vision Statement .....                                                     | 1  |
| 1.3 Problem Statement .....                                                    | 1  |
| 1.4 Problem Solution .....                                                     | 2  |
| 1.5 Objectives of the Proposed System .....                                    | 2  |
| 1.6 Scope .....                                                                | 3  |
| 1.6.1 Limitations/Constraints .....                                            | 3  |
| 1.7 Modules .....                                                              | 3  |
| 1.7.1 Module 1: User Profile & Settings .....                                  | 3  |
| 1.7.2 Module 2: Voice Command Interface .....                                  | 4  |
| 1.7.3 Module 3: Pantry Management .....                                        | 4  |
| 1.7.4 Module 4: Shopping Assistant & Product Recognition .....                 | 4  |
| 1.7.5 Module 5: Emergency Management & Unusual Activity Detection .....        | 4  |
| 1.7.6 Module 6: Volunteer Mode .....                                           | 4  |
| 1.8 Related System Analysis/Literature Review .....                            | 5  |
| 1.8.1 Literature Review .....                                                  | 5  |
| 1.8.2 Related System Analysis .....                                            | 5  |
| 1.9 Tools and Technologies .....                                               | 6  |
| 1.10 Project Contribution .....                                                | 7  |
| 1.11 Relevance to Course Modules .....                                         | 8  |
| 2 Chapter 2: Requirement Analysis .....                                        | 9  |
| 2.1 User Classes and Characteristics .....                                     | 9  |
| 2.1.1 Use case Diagram .....                                                   | 10 |
| 2.2 Functional Requirements .....                                              | 11 |
| Mockup M1-Onboarding & Profile Setup Screen .....                              | 11 |
| Functional Requirements Derived from Mockup M1 - Login Screen .....            | 12 |
| Mockup M2 - Dashboard Screen .....                                             | 12 |
| Functional Requirements Derived from Mockup M2 - Dashboard Screen .....        | 12 |
| Mockup M3 - Pantry Management Screen .....                                     | 13 |
| Functional Requirements Derived from Mockup M3 - Pantry Management .....       | 13 |
| Mockup M4 - Shopping & Product Recognition Screen .....                        | 14 |
| Functional Requirements Derived from Mockup M4 - Shopping Screen .....         | 14 |
| Mockup M5 - Object Detection Screen .....                                      | 15 |
| Functional Requirements Derived from Mockup M5 - Object Detection Screen ..... | 15 |
| Mockup M6 - Emergency Management Screen .....                                  | 16 |
| Functional Requirements Derived from Mockup M6 - Emergency Screen .....        | 16 |
| Mockup M7 - Volunteer Mode Screen .....                                        | 17 |
| Functional Requirements Derived from Mockup M7- Volunteer Mode Screen .....    | 17 |
| 2.3 Non-Functional Requirements .....                                          | 19 |
| 2.3.1 Reliability .....                                                        | 19 |
| 2.3.2 Usability .....                                                          | 19 |
| 2.3.3 Performance .....                                                        | 19 |
| 2.3.4 Security .....                                                           | 19 |
| 2.4 External Interface Requirements .....                                      | 20 |
| 2.4.1 User Interfaces Requirements .....                                       | 20 |
| 2.4.2 Software Interfaces .....                                                | 20 |

|       |                                                                                       |    |
|-------|---------------------------------------------------------------------------------------|----|
| 2.4.3 | Hardware Interfaces .....                                                             | 21 |
| 2.4.4 | Communications Interfaces.....                                                        | 21 |
| 3     | Chapter 3: Design and Architecture.....                                               | 22 |
| 3.1   | System Architecture Overview .....                                                    | 22 |
| 3.1.1 | Purpose .....                                                                         | 22 |
| 3.2   | Conceptual Architecture Diagram.....                                                  | 23 |
| 3.2.1 | Technologies and Services .....                                                       | 24 |
| 3.3   | Architecture Style / Pattern .....                                                    | 25 |
| 3.4   | Design Models .....                                                                   | 27 |
| 3.4.1 | Design Models for Object Oriented Development Approach .....                          | 27 |
| 3.5   | Data Design.....                                                                      | 40 |
| 3.5.1 | Data Representation and Structure .....                                               | 40 |
|       | User Profile Data: .....                                                              | 40 |
|       | Pantry and Inventory Data: .....                                                      | 40 |
| 3.5.2 | Storage, Processing, and Organization of Major Entities .....                         | 42 |
| 3.5.3 | Databases and Data Storage Items .....                                                | 42 |
| 4     | Chapter 4: Implementation .....                                                       | 44 |
| 4.1   | Project Methodology & Algorithms .....                                                | 44 |
| 4.1.1 | Project Methodology (Step-by-Step Approach).....                                      | 44 |
| 4.1.2 | Algorithm .....                                                                       | 46 |
| 4.2   | Training Results & Model Evaluation .....                                             | 48 |
| 4.2.1 | M-01: SEEVIA-LOCAL-PRODUCT-RECOGNIZER (YOLOv8-Nano).....                              | 48 |
| 4.2.2 | M-02: SEEVIA-ZEROSHOT-EXTRACTOR (DistilBERT NER).....                                 | 50 |
| 4.2.3 | M-03: SEEVIA-INTENT-MODEL-IMPROVED (FastText/CNN) .....                               | 51 |
| 4.2.4 | M-04: SEEVIA-FALL-DETECTOR (SVM / Random Forest) .....                                | 52 |
| 4.2.5 | M-05: SEEVIA-AISLE-NAVIGATION-DQN (Deep Q-Network) .....                              | 53 |
| 4.2.6 | Consolidated Model Performance Summary .....                                          | 53 |
| 4.3   | Security Techniques.....                                                              | 55 |
| 4.3.1 | Authentication: .....                                                                 | 55 |
| 4.3.2 | Authorization: .....                                                                  | 55 |
| 4.3.3 | Data Privacy: .....                                                                   | 55 |
| 4.4   | External APIs/SDKs .....                                                              | 55 |
| 4.5   | User Interface.....                                                                   | 59 |
| 4.5.1 | Role PWD: .....                                                                       | 59 |
| 4.5.2 | Role Caregiver: .....                                                                 | 66 |
| 4.5.3 | Role Volunteer: .....                                                                 | 67 |
| 4.6   | Deployment.....                                                                       | 69 |
| 5     | Chapter 5: Testing and Evaluation.....                                                | 70 |
| 5.1   | Unit Testing .....                                                                    | 70 |
| 5.2   | Functional Testing .....                                                              | 72 |
| 5.3   | Business Rules Testing .....                                                          | 73 |
| 5.4   | Integration Testing.....                                                              | 74 |
|       | Integration Test 1: M-01 + M-02 → Pantry Firestore.....                               | 74 |
|       | Integration Test 2: M-04 (SEEVIA-FALL-DETECTOR) → Emergency → Volunteer Dispatch .... | 74 |
|       | Integration Test 3: M-03 (Intent) → M-05 (Navigation) → M-01 (Detection) .....        | 75 |
| 6     | Chapter 6: Conclusion and Future Work .....                                           | 76 |
| 6.1   | Conclusion .....                                                                      | 76 |
| 6.2   | Future Work.....                                                                      | 76 |

|   |                                             |    |
|---|---------------------------------------------|----|
| 7 | Chapter 7: References .....                 | 78 |
| 8 | Chapter 8: Plagiarism Report.....           | 79 |
|   | Appendix D.....                             | 80 |
|   | General Coding Standards & Guidelines ..... | 80 |

# List of Figures

|                                                                                         |    |
|-----------------------------------------------------------------------------------------|----|
| Figure 1 Use Case Diagram of Seevia .....                                               | 10 |
| Figure 2 Mockup M1 - Onboarding & Profile Setup Screen.....                             | 11 |
| Figure 3 Mockup M2 - Dashboard Page.....                                                | 12 |
| Figure 4 Mockup M3 - Pantry Management Screen.....                                      | 13 |
| Figure 5 Mockup M4 - Shopping Assistant & Object Detection Screen.....                  | 14 |
| Figure 6 Mockup M5 - Object Detection Screen.....                                       | 15 |
| Figure 7 Mockup M6 - Emergency Management Screen.....                                   | 16 |
| Figure 8 Mockup M7 - Volunteer Mode Screen.....                                         | 17 |
| Figure 9 High-Level Conceptual Architecture for Seevia .....                            | 23 |
| Figure 10 Components Layered Architecture.....                                          | 26 |
| Figure 11 AD-01: Activity Diagram User Onboarding & Preferences Setup .....             | 27 |
| Figure 12 AD-02: Activity Diagram of Voice Command Handling .....                       | 28 |
| Figure 13 AD-03: Activity Diagram of Pantry Management & Shopping Assistance.....       | 29 |
| Figure 14 AD-04: Activity Diagram of Emergency SOS & Fall Detection .....               | 30 |
| Figure 15 AD-05: Activity Diagram of Volunteer Emergency Response.....                  | 31 |
| Figure 16 AD-05: Activity Diagram of Volunteer Emergency Response.....                  | 32 |
| Figure 17 AD-06: Activity Diagram of In-Store Product Identification & Navigation ..... | 33 |
| Figure 18 CD-01: Class Diagram of Seevia .....                                          | 34 |
| Figure 19 -SD-01: Sequence Diagram of Voice Command Flow .....                          | 35 |
| Figure 20 SD-02:Sequence Diagram of Product Recognition Flow.....                       | 36 |
| Figure 21 SD-03: Sequence Diagram of Pantry Management Flow .....                       | 37 |
| Figure 22 SD-04: Sequence Diagram of Shopping Assistant Flow.....                       | 38 |
| Figure 23 SD-05: Sequence Diagram of Emergency SOS Alert Flow .....                     | 39 |
| Figure 24 - Product Recognition Performance Graph .....                                 | 49 |
| Figure 25 - Product Recognition Loss Graph .....                                        | 50 |
| Figure 26 SEEVIA-ZEROSHOT-EXTRACTOR NER Precision/Recall .....                          | 50 |
| Figure 27 SEEVIA-INTENT-MODEL Confusion Matrix .....                                    | 51 |
| Figure 28 SEEVIA-FALL-DETECTOR ROC Curve .....                                          | 52 |
| Figure 29 SEEVIA-AISLE-NAVIGATION-DQN Training.....                                     | 53 |
| Figure 30 Auth and Onboarding Screen .....                                              | 60 |
| Figure 31 Pantry Manager UI .....                                                       | 61 |
| Figure 32 Product Recognition UI.....                                                   | 62 |
| Figure 33 Emergency Screen .....                                                        | 63 |
| Figure 34 Navigation Screen .....                                                       | 64 |
| Figure 35 Map Learning Screen .....                                                     | 65 |
| Figure 36 Shopping Screen.....                                                          | 66 |
| Figure 37 Caregiver Screens .....                                                       | 67 |
| Figure 38 Volunteer Screens.....                                                        | 68 |

# List of Tables

|                                                                                             |    |
|---------------------------------------------------------------------------------------------|----|
| Table 1.1 Related System Analysis with proposed project solution                            | 5  |
| Table 1.2 Tools and Technologies for Proposed Project                                       | 6  |
| Table 2.1 Shows user classes and character                                                  | 9  |
| Table 2.2 Functional Requirements Derive from Mockup M1                                     | 12 |
| Table 2.3 Functional Requirements Derive from Mockup M2                                     | 12 |
| Table 2.4 Functional Requirements Derive from Mockup M3                                     | 13 |
| Table 2.5 Functional Requirements Derive from Mockup M4                                     | 14 |
| Table 2.6 Functional Requirements Derive from Mockup M5                                     | 15 |
| Table 2.7 Functional Requirements Derive from Mockup M6                                     | 16 |
| Table 2.8 Functional Requirements Derive from Mockup M7                                     | 17 |
| Table 2.9 Event-Response Table for Seevia                                                   | 18 |
| Table 3.1 Technologies, Components, and Security Mechanisms Used in the System Architecture | 24 |
| Table 4.1 Algorithms                                                                        | 46 |
| Table 4.3 : M-01 Training Configuration                                                     | 48 |
| Table 4.4 M-01 Performance Metrics                                                          | 49 |
| Table 4.5 M-02 SEEVIA-ZEROSHOT-EXTRACTOR Performance                                        | 50 |
| Table 4.6 M-03 SEEVIA-INTENT-MODEL-IMPROVED Performance                                     | 51 |
| Table 4.7 M-04 SEEVIA-FALL-DETECTOR Performance                                             | 52 |
| Table 4.8 M-05 SEEVIA-AISLE-NAVIGATION-DQN Performance                                      | 53 |
| Table 4.9 : Consolidated AI Model Performance Summary                                       | 53 |
| Table 4.10 Details of APIs used in the project                                              | 55 |
| Table 4.11 Seevia Deployment Configuration                                                  | 69 |
| Table 5.1 Unit Test 1 — SEEVIA-INTENT-MODEL                                                 | 70 |
| Table 5.2 Unit Test 2 — SEEVIA-LOCAL-PRODUCT-RECOGNIZER                                     | 70 |
| Table 5.3 Unit Test 3 — SEEVIA-ZEROSHOT-EXTRACTOR                                           | 71 |
| Table 5.4 Unit Test 4 — SEEVIA-FALL-DETECTOR                                                | 71 |
| Table 5.5 Unit Test 5 — SEEVIA-AISLE-NAVIGATION-DQN                                         | 71 |
| Table 5.6 User Authentication Test                                                          | 72 |
| Table 5.7 Pantry Test                                                                       | 72 |
| Table 5.8 Product Recognition Test                                                          | 72 |
| Table 5.9 AI Tier Routing Decision Table                                                    | 73 |
| Table 5.10 Emergency Escalation Decision Table                                              | 73 |
| Table 5.11 Pantry Scan Routing Rules                                                        | 74 |
| Table 5.12 Integration Test 1                                                               | 74 |
| Table 5.13 Integration Test 2                                                               | 75 |
| Table 5.14 Integration Test 3                                                               | 75 |

# 1 Chapter 1: Introduction and Problem Definition

This chapter introduces Seevia, an AI-powered voice assistant designed to support visually impaired individuals in performing daily tasks independently. It outlines the project's motivation, problem scope, objectives, and the need for an intelligent, accessible solution that enhances autonomy and safety through technology.

## 1.1 Overview of the Project

Seevia is an AI-powered, voice-first mobile application designed to assist visually impaired individuals in performing everyday tasks independently. The project operates within the assistive technology and AI accessibility domain, targeting users who face barriers in managing household activities, shopping, and navigation due to visual impairments. The system bridges the accessibility gap by providing a smart, voice-driven assistant that integrates multiple essential functions, including object recognition, pantry management, shopping assistance, emergency alerts, and volunteer coordination, into one intuitive platform. By combining voice interaction, computer vision, and machine learning, Seevia empowers users to interact naturally with their environment through speech and camera inputs, providing real-time audio and haptic feedback at every step. The proposed system reduces dependency on caregivers and replaces fragmented assistive tools with a unified, affordable, and intelligent solution. Built on React Native with Expo for cross-platform Android deployment and Firebase for real-time cloud synchronization, the application serves visually impaired users, their caregivers, and community volunteers through a coherent three-role architecture. A three-tier AI fallback strategy, from on-device TensorFlow Lite models through HuggingFace Inference API to cloud services, ensures continuous operation even in low-connectivity conditions. In doing so, Seevia enhances autonomy, confidence, and quality of life for visually impaired Pakistanis, offering not only practical assistance but also emotional empowerment through accessible technology.

## 1.2 Vision Statement

**For** visually impaired individuals **who** face challenges in performing daily tasks independently due to limited access to consistent and intelligent assistance, **the** Seevia **is** an AI-powered smart assistant application **that** provides voice-controlled task execution, object and product recognition, shopping and pantry management, and emergency support. **Unlike** solutions that rely on human agents, volunteers, or expensive hardware, **our product** (Seevia) offers an affordable, fully autonomous, and integrated AI solution through smartphones, empowering users to navigate daily life with confidence and independence.

## 1.3 Problem Statement

Millions of visually impaired individuals face significant challenges in performing daily tasks independently, such as recognising products, managing groceries, or navigating unfamiliar environments. In Pakistan, over 1.8 million people live with some form of visual impairment, many of whom lack access to effective assistive technologies due to financial, technical, or

infrastructural limitations. Existing solutions are often expensive, fragmented, or fail to provide real-time, voice-based support in Urdu, leaving users dependent on caregivers for routine activities such as reading product labels, checking expiry dates, navigating store aisles, and responding to health emergencies.

Internationally available systems such as Seeing AI (English-only), Be My Eyes (volunteer-dependent), Aira (subscription-based), and OrCam MyEye (expensive dedicated hardware) all fail to address the specific socio-economic, linguistic, and infrastructural context of Pakistani users. There remains a critical need for an accessible, inclusive, adaptive, and locally-contextualised technology solution that can effectively and affordably support the everyday lives of visually impaired individuals in Pakistan.

## **1.4 Problem Solution**

Seevia resolves these challenges through five domain-specific fine-tuned AI models deployed on an affordable Android smartphone. The SEEVIA-INTENT-MODEL-IMPROVED model, built on FastText and a custom CNN architecture, interprets natural language commands in Roman Urdu and English, mapping complex bilingual vernacular to five core system intents in real time. The SEEVIA-LOCAL-PRODUCT-RECOGNIZER (YOLOv8-Nano, fine-tuned on 5,722 images of Pakistani grocery products) identifies items through the camera and triggers the SEEVIA-ZEROSHOT-EXTRACTOR (fine-tuned DistilBERT NER) to parse raw OCR text from packaging labels, extracting Brand Name, Expiry Date, and Allergen entities even from locally-branded products such as Shan, Olper's, and Nestlé Pakistan variants.

In-store navigation is handled by the SEEVIA-AISLE-NAVIGATION-DQN, a Deep Q-Network fine-tuned with a human-centric reward function that penalises 90-degree turns and shelf collisions, producing natural straight-path guidance. Safety is maintained by the SEEVIA-FALL-DETECTOR, an SVM/Random Forest anomaly detection model that continuously monitors accelerometer and gyroscope data, distinguishing genuine falls from phone drops through carefully tuned threshold logic that minimises caregiver false alarm fatigue. All five models escalate to HuggingFace APIs and then Gemini 1.5 Flash when confidence falls below 0.75, ensuring high-reasoning validation for ambiguous inputs.

## **1.5 Objectives of the Proposed System**

BO-1: Enable visually impaired individuals to perform daily tasks independently using an AI-powered voice assistant.

BO-2: Enhance user autonomy by integrating personalized features for pantry management, product recognition, and shopping assistance.

BO-3: Improve user safety by providing real-time fall detection, automated emergency alerts, and easy access to volunteer assistance.

BO-4: Reduce dependence on caregivers for routine activities by delivering an offline, capable, self-sufficient AI platform.

BO-5: Achieve product recognition accuracy of at least 85% under standard lighting using YOLOv8-Nano trained on Pakistani product datasets.

BO-6: Improve the overall quality of life for visually impaired individuals by providing a comprehensive, WCAG 2.1 AA compliant mobile platform.

## 1.6 Scope

The scope of Seevia is to design and develop an AI-powered mobile application tailored specifically for visually impaired individuals to assist them in performing daily activities independently and efficiently. The application includes six integrated modules addressing essential aspects of everyday life through voice-controlled interaction, camera-based recognition, and intelligent decision-making. Core functionalities include a voice command interface for hands-free operation in Urdu and English; a personalized user profile storing preferences, health data, and emergency contacts; a pantry and shopping management module for barcode scanning and expiry tracking; a shopping assistant with real-time product identification and reinforcement learning navigation; an emergency alert mechanism supporting voice trigger, gesture, and automated fall detection; and a volunteer network for community-based emergency response. The application is optimized for offline functionality for critical tasks and designed for compatibility with low, to mid-range Android smartphones for broader accessibility. Deep learning models (YOLOv8-Nano), natural language processing, and reinforcement learning power the core AI features. The system boundary excludes general-purpose outdoor navigation in the style of Google Maps and complex medical diagnostics. Integration with third-party services includes Firebase, Deepgram, Google Vision API, Open Food Facts API, and Twilio SMS. The final deliverable is a fully functional prototype validated with real-world Pakistani product datasets and user testing, and automated fall detection; and a volunteer network for community-based emergency response.

### 1.6.1 Limitations/Constraints

LI-1: The system relies on smartphone hardware (camera, microphone, internet), which may limit performance for users with older or low-end devices.

LI-2: Object detection and product recognition accuracy may be affected by poor lighting or low-resolution camera images.

LI-3: The AI assistant has limited performance in offline mode; certain cloud-dependent features such as Deepgram STT require internet connectivity.

LI-4: Initial training datasets may not cover all regional products or household items- reducing recognition accuracy for uncommon items.

LI-5: Emergency alert functionality depends on mobile network or internet connectivity to notify contacts and volunteers.

LI-6: The application currently supports Urdu and English only, limiting usability for speakers of other regional languages.

## 1.7 Modules

### 1.7.1 Module 1: User Profile & Settings

FE-1: Create and manage user profiles with voice-guided onboarding and personalization.

FE-2: Customize app settings including voice pitch, command sensitivity, language, and accessibility preferences (e.g., high-contrast mode, voice-only interaction).

FE-3: Store and manage basic health information such as medications, allergies, and emergency notes.

### **1.7.2 Module 2: Voice Command Interface**

FE-1: Activate and control all app features using voice commands.

FE-2: Provide real-time voice feedback and prompts for user queries.

FE-3: Interpret natural language inputs for better user interaction.

FE-4: Operate essential voice commands in offline mode using on-device speech processing.

### **1.7.3 Module 3: Pantry Management**

FE-1: Add items to pantry using barcode scanning or voice input.

FE-2: Track item expiry dates and notify users.

FE-3: Auto-generate shopping lists based on low-stock or expired pantry items.

FE-4: Allow voice-based edits and list management without screen interaction.

FE-5: Recommend or restrict pantry items based on user-defined dietary preferences or allergies stored in the profile.

### **1.7.4 Module 4: Shopping Assistant & Product Recognition**

FE-1: Identify products using camera-based object detection and OCR.

FE-2: Provide product details (name, category, brand, expiry) via voice feedback.

FE-3: Assist in finding similar or alternative items.

FE-4: Guide users through shopping process using real-time voice feedback and reinforcement learning navigation.

FE-5: Learn and store layout maps of shopping centers for improved navigation on return visits.

### **1.7.5 Module 5: Emergency Management & Unusual Activity Detection**

FE-1: Detect and respond to unusual activities (e.g., long periods of inactivity, falls, or abnormal movement patterns) using sensor input and contextual analysis.

FE-2: Send emergency alerts to predefined contacts with GPS location via voice command, gesture, or automatic detection.

FE-3: Trigger auditory or vibration-based alarms manually or through predefined voice gestures for instant assistance.

FE-4: Provide real-time voice instructions for first responders based on stored health info.

FE-5: Auto-escalate to volunteer community or services if primary contact is unreachable.

### **1.7.6 Module 6: Volunteer Mode**

FE-1: Enable community members to register as volunteers for emergency assistance with identity verification.

FE-2: Notify the nearest available volunteer if the user's emergency contacts are unresponsive.

FE-3: Display proximity-based active cases, allowing volunteers to accept or decline based on availability.

FE-4: Allow volunteers to enhance Seevia's product and environment knowledge by verifying product data and mapping environments.

FE-5: Include a volunteer rating and reputation system to maintain trust and quality of assistance.

## 1.8 Related System Analysis/Literature Review

### 1.8.1 Literature Review

Recent research in assistive technologies for visually impaired individuals has demonstrated significant progress in computer vision, speech recognition, and wearable aids, while also revealing persistent integration gaps. Studies by *Lopes et al. (2022)* and *Malik (2020)* emphasize that AI-driven interfaces substantially enhance independence for visually impaired users; however, most existing solutions address only single functions such as navigation or text reading rather than integrated daily-living support. Tools like Seeing AI and Be My Eyes depend heavily on internet connectivity or human assistance, limiting real-time autonomy in low-resource settings.

Emerging projects such as the AI Smart Cane (*Wazirali et al., 2025*) apply edge AI for navigation and obstacle detection, improving mobility but lacking integrated support for household inventory management, shopping assistance, or emergency response. Similarly, OCR-based reading assistants focus on text-to-speech conversion but do not incorporate context-aware pantry management or fall detection. Research by *Jurafsky and Martin (2023)* on speech and language processing provides the theoretical basis for Seevia's NLP pipeline, while *Goodfellow et al. (2016)* on deep learning informs the architecture of the YOLOv8 product recognition and LSTM-based anomaly detection components.

Overall, current systems remain fragmented, expensive, or dependent on external human aid. There is a clear research and usability gap for an affordable, all-in-one AI assistant that supports multiple aspects of daily living through a voice-first Urdu interface. Seevia directly addresses this gap by integrating computer vision, NLP, reinforcement learning, and sensor fusion into one adaptive, offline-capable mobile platform designed specifically for visually impaired users in Pakistan.

### 1.8.2 Related System Analysis

Existing systems for visually impaired individuals offer partial solutions but often have limitations in terms of functionality, accessibility, or cost. While some focus on navigation or product recognition, they lack integration of essential daily tasks like shopping assistance, pantry management, and emergency support. Others rely on external help, leading to variability in service. Seevia aims to provide a comprehensive, autonomous AI-powered solution that addresses these gaps and enhances user independence.

Table 1.1 Related System Analysis with proposed project solution

| Application Name | Weakness | Proposed Project Solution |
|------------------|----------|---------------------------|
|------------------|----------|---------------------------|

|             |                                                                                        |                                                                                                                                                                 |
|-------------|----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Aira        | Relies on human agents, subscription-based service, can be costly and less accessible. | Seevia is fully AI-powered, removing the need for human agents, offering cost-effective, autonomous solutions.                                                  |
| Be My Eyes  | Depends on volunteers, leading to variability in service availability.                 | Seevia eliminates the need for external volunteers, offering a reliable and continuous AI-powered assistant.                                                    |
| Seeing AI   | Lacks integration for household management and emergency support.                      | Seevia combines AI, voice control, and camera recognition to offer shopping assistance, pantry management, and emergency alerts in one platform.                |
| TellBack    | Focuses only on text-to-speech and basic navigation, lacking task-specific tools.      | Seevia enhances TalkBack's functionality by adding product recognition, personalized task management, and emergency support through an integrated AI assistant. |
| OrCam MyEye | Expensive hardware solution, not accessible to all visually impaired users.            | Seevia provides a more affordable software solution through smartphones, making it accessible to a larger group of users.                                       |

## 1.9 Tools and Technologies

Seevia will be built using modern development tools and intelligent systems. This includes technologies for model development, voice processing, computer vision, reinforcement learning, and cross-platform mobile development.

**Table 1.2 Tools and Technologies for Proposed Project**

| Tools / Technologies |                    | Purpose                                                                  | Rationale                                                   |
|----------------------|--------------------|--------------------------------------------------------------------------|-------------------------------------------------------------|
| <b>Tools</b>         | Visual Studio Code | Used for writing, debugging, and editing code across multiple languages. | Lightweight, extensible, and supports multiple frameworks.  |
|                      | Firebase           | Provides real-time database, authentication, and cloud storage.          | Ensures scalability, low latency, and secure data handling. |
|                      | Figma              | Used to create UI/UX mockups and design prototypes.                      | Enables collaborative and accessible interface design.      |

|                     |                             |                                                                         |                                                                             |
|---------------------|-----------------------------|-------------------------------------------------------------------------|-----------------------------------------------------------------------------|
|                     | GitHub                      | Repository management and collaboration.                                | Ensures code backup, version tracking, and teamwork efficiency.             |
| <b>Technologies</b> | Python                      | Used for backend logic and AI model development.                        | Provides rich libraries for AI, ML, and NLP.                                |
|                     | React                       | Used for mobile app development and cross-platform web interface.       | Cross-platform, efficient, and supports modular, responsive UI development. |
|                     | TensorFlow Lite             | Used for deploying AI models on mobile devices.                         | Lightweight and optimized for on-device inference.                          |
|                     | OpenCV                      | Enables camera-based object detection and computer vision features.     | Efficient, open-source library for real-time image processing.              |
|                     | spaCy                       | Used for natural language processing tasks.                             | Provides fast and accurate NLP pipelines.                                   |
|                     | Hugging Face Transformers   | Pre-trained models for NLP tasks such as speech and intent recognition. | Enables efficient integration of state-of-the-art language models.          |
|                     | Stable-Baselines3           | Used for reinforcement learning model training.                         | Provides robust RL algorithms for adaptive learning.                        |
|                     | Google ML Kit               | Provides on-device OCR for text and product label reading.              | Offline and privacy-preserving mobile ML toolkit.                           |
|                     | Flutter TTS / STT Plugins   | Enables text-to-speech and speech-to-text interactions.                 | Provides real-time voice communication with users.                          |
|                     | Flutter Provider / Riverpod | Manages app state efficiently across screens.                           | Improves app performance and maintainability.                               |
|                     | TensorFlow Hub              | Provides pre-trained models for transfer learning.                      | Reduces training time and improves model accuracy.                          |

## 1.10 Project Contribution

The Seevia system introduces multiple technical and conceptual innovations designed to advance assistive technology for visually impaired users in Pakistan. Unlike existing fragmented or costly solutions, Seevia integrates artificial intelligence, NLP, and computer vision into one unified, locally contextualized, affordable mobile platform.

## Key Contributions:

- **AI-Driven Voice Interaction:** Seevia leverages advanced NLP to interpret and respond to mixed Urdu/English voice commands in real-time, enabling fully voice-controlled app navigation and task execution, the first such system designed for Pakistani users.
- **Context-Aware Object Recognition:** The system uses YOLOv8-Nano with OpenCV and Google ML Kit OCR to identify everyday products, read labels and expiry dates, and provide audio feedback, trained specifically on 23 Pakistani product classes.
- **Smart Pantry Management:** Introduces automated grocery tracking, expiry date alerts, and AI-generated shopping lists through camera-based recognition and voice input, significantly reducing caregiver dependency for household management.
- **Adaptive Learning through Reinforcement Models:** Seevia's Stable-Baselines3 RL component allows the system to learn store layouts from repeated visits, improving navigation guidance accuracy over time, and personalizing the shopping experience.
- **Emergency Detection and Volunteer Network Integration:** Incorporates sensor fusion (accelerometer + gyroscope) for automated fall detection combined with a community volunteer dispatch system, providing a safety net unavailable in any existing Pakistani assistive application.
- **Multilingual and Accessibility-Optimized Interface:** Designed with Urdu and English support, WCAG 2.1 AA high-contrast UI, and full screen-reader compatibility, making Seevia the most accessible AI platform for Pakistani PWDs.

## Contribution Impact:

By combining AI, NLP, computer vision, and reinforcement learning within a single application, Seevia transforms assistive technology into a proactive digital companion. Its innovation lies in unifying critical daily functionalities, object identification, voice control, emergency support, and behavioral learning, into an affordable and localized platform that significantly improves autonomy, safety, and quality of life for visually impaired Pakistanis.

## 1.11 Relevance to Course Modules

- **Machine Learning:** Used reinforcement learning (StableBaselines3) for store navigation and supervised learning (YOLOv8) for product recognition with transfer learning from COCO pretrained weights.
- **Natural Language Processing (NLP):** Implemented complete NLP pipeline from STT to intent classification using spaCy and Hugging Face Transformers for real-time Urdu/English command understanding.
- **Computer Vision (CV):** Integrated YOLOv8-Nano and OpenCV for real-time object detection, Google ML Kit for OCR, and camera-based product scanning.
- **Mobile App Development:** Developed a cross-platform mobile application using React Native with Expo, Expo Router for file-based navigation, and Firebase SDK for backend integration.
- **Database Systems:** Utilized Firestore for real-time data storage, user authentication, and secure cross-device synchronization with role-based Firestore security rules.

## 2 Chapter 2: Requirement Analysis

This chapter describes the requirements, system actors, and functionalities of Seevia. It identifies user roles, functional and non-functional requirements, system interfaces, and diagrammatic representations through use-case diagrams and mockup-based requirement analysis.

### 2.1 User Classes and Characteristics

This section identifies the primary user classes that will interact with the Seevia system and outlines their key roles, interaction modes, and characteristics.

Table 2.1 Shows user classes and character

| User class                    | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Visually Impaired User</b> | The primary user of Seevia. Uses the application as a personal AI assistant to manage daily tasks including product identification, pantry tracking, shopping guidance, and emergency alerts. Relies exclusively on voice commands and audio feedback for all interactions, no visual input is required. Estimated usage: 3-5 interactions per day for daily-living support. Users vary in technical literacy, so Seevia prioritizes a minimal, voice-first interface with haptic feedback at every step. |
| <b>Volunteer</b>              | Community members or registered individuals who assist visually impaired users during emergencies or on-demand help requests. Volunteers view nearby help requests, respond to SOS alerts, and provide situational guidance via the Seevia app. They require internet access and GPS location services for proximity-based task assignment. Onboarding material is provided within the app, and a rating system ensures quality of service.                                                               |
| <b>Emergency Responder</b>    | Caregivers, family members, or emergency services (Rescue 1122, Edhi Foundation) who receive SOS alerts through the integrated emergency module. Their interaction with Seevia is limited to receiving user location, health summary, and emergency type. These users require accurate and immediate data from Seevia's Firebase backend to respond effectively.                                                                                                                                          |
| <b>AI Engine</b>              | The AI engine interprets voice commands, processes natural language intent, and identifies products through camera input and OCR. Although not a human actor, it plays a central role in maintaining Seevia's voice-first functionality. It interfaces with TFLite models, Hugging Face APIs, and cloud services based on confidence-threshold routing.                                                                                                                                                   |

## 2.1.1 Use case Diagram

The following use case diagram provides an overview of all Seevia modules.

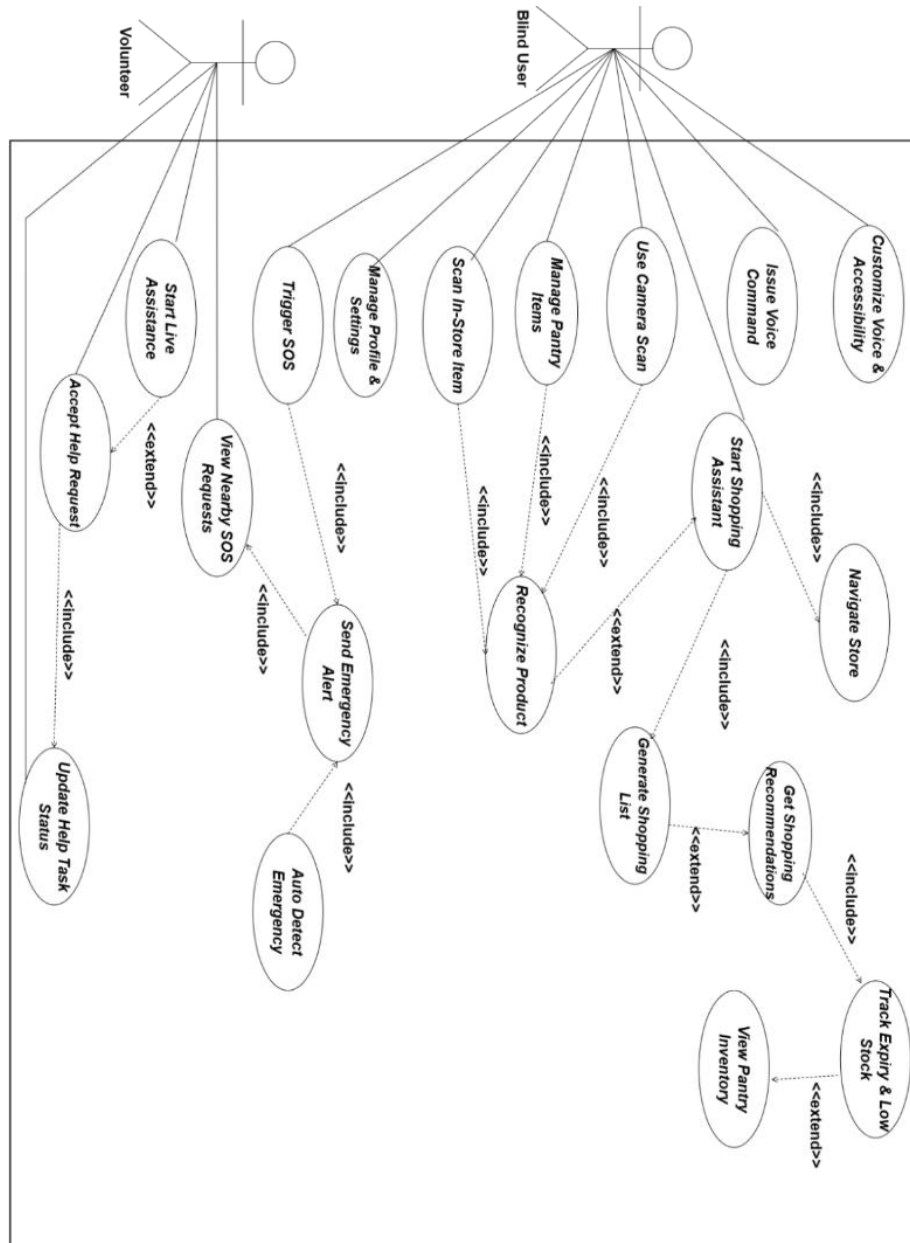


Figure 1 Use Case Diagram of Seevia

## 2.2 Functional Requirements

This section presents a structured application of the Mockup-Based Requirement Analysis technique for the Seevia application. It demonstrates how functional requirements are derived directly from user interface elements and interactions designed for visually impaired users. Each mockup corresponds to a key module in Seevia’s voice-first system, emphasizing accessibility and AI integration.

### Mockup M1-Onboarding & Profile Setup Screen

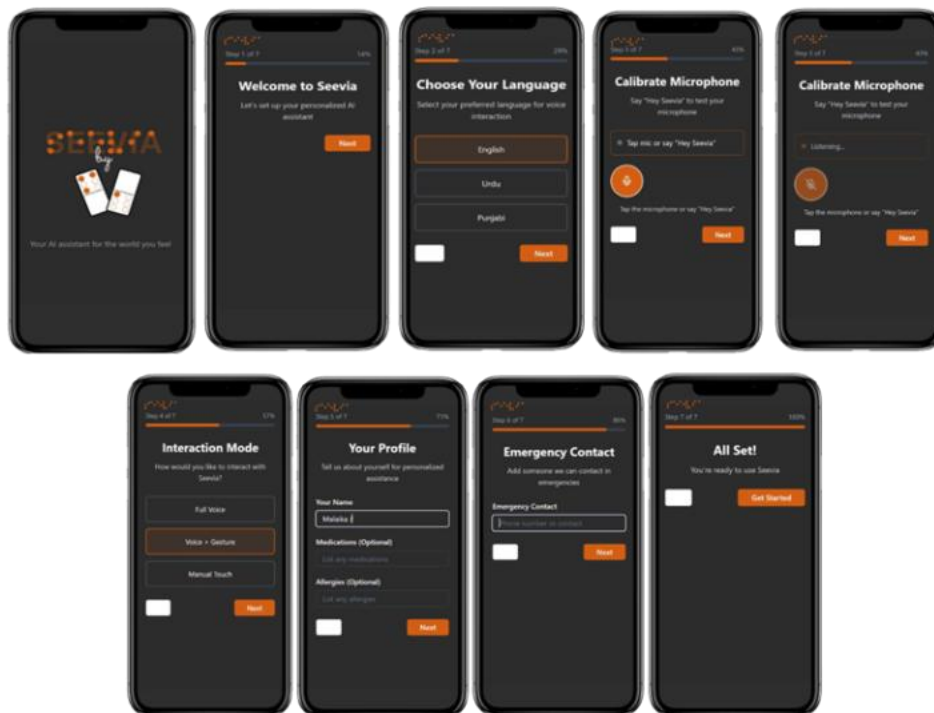


Figure 2 Mockup M1 - Onboarding & Profile Setup Screen

Initial setup includes voice-guided onboarding where the user is asked simple questions. Preferences such as language, voice pitch, and emergency contacts are saved. This setup is completed once per user. All fields are accessible via microphone input or the wake phrase “**Hey Seevia.**”

## Functional Requirements Derived from Mockup M1 - Login Screen

Table 2.2 Functional Requirements Derive from Mockup M1

| Feature                   | Functional Requirement                                                                                                   | Business Rule                                                                    |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Voice-Guided Setup        | FR1.1: The system shall guide the user through voice-based onboarding to collect profile information.                    | All onboarding prompts must have both voice and tactile accessibility options.   |
| Profile Creation          | FR1.2: The system shall allow users to create a personalized profile storing language, voice, and emergency preferences. | Each user profile is stored securely in Firebase and linked to a unique user ID. |
| Emergency Contacts        | FR1.3: The system shall allow users to add, edit, or remove emergency contact details.                                   | Minimum one contact must be stored before onboarding completes.                  |
| Accessibility Preferences | FR1.4: The system shall save selected voice pitch, contrast mode, and interaction preferences.                           | Preferences persist across sessions until manually changed.                      |

## Mockup M2 - Dashboard Screen

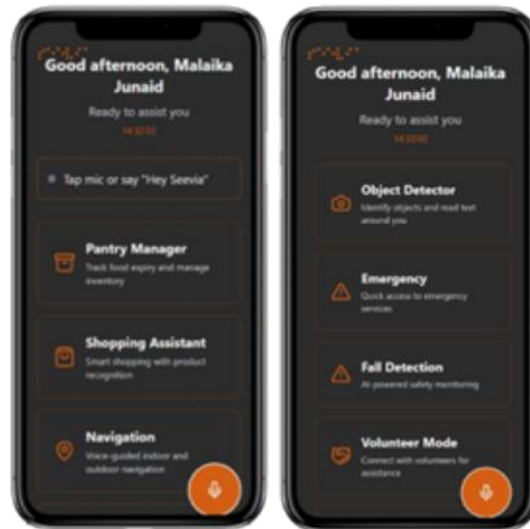


Figure 3 Mockup M2 - Dashboard Page

The main dashboard allows hands-free navigation across modules. Users can say “**Hey Seevia, open pantry**” or tap any area for voice feedback and selection confirmation.

## Functional Requirements Derived from Mockup M2 - Dashboard Screen

Table 2.3 Functional Requirements Derive from Mockup M2

| Feature | Functional Requirement | Business Rule |
|---------|------------------------|---------------|
|---------|------------------------|---------------|

|                            |                                                                                                                |                                                                |
|----------------------------|----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| Voice-Activated Navigation | FR2.1: The system shall enable users to access all modules via predefined voice commands.                      | Commands must be recognized in English and Urdu.               |
| Dashboard Menu             | FR2.2: The dashboard shall display accessible shortcuts to Pantry, Shopping, Emergency, and Volunteer modules. | Icons must be labeled with alt-text and voice tags.            |
| Real-Time Feedback         | FR2.3: The system shall provide real-time voice feedback after each selection.                                 | Feedback must be contextually relevant and short (<3 seconds). |
| Voice Wake Command         | FR2.4: The system shall recognize “Hey Seevia” as a wake phrase even when the screen is off.                   | Wake phrase detection runs on-device to save battery.          |

## Mockup M3 - Pantry Management Screen

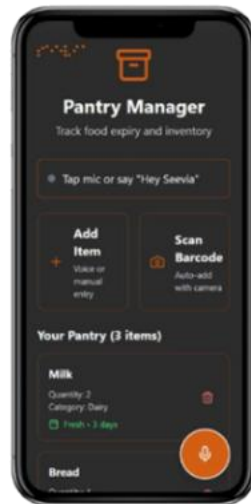


Figure 4 Mockup M3 - Pantry Management Screen

Users can add items by saying “**Hey Seevia, add milk**” or scanning barcodes. Navigation through pantry categories is fully voice-enabled with automatic audio feedback.

## Functional Requirements Derived from Mockup M3 - Pantry Management

Table 2.4 Functional Requirements Derive from Mockup M3

| Feature         | Functional Requirement                                                                   | Business Rule                                       |
|-----------------|------------------------------------------------------------------------------------------|-----------------------------------------------------|
| Add Pantry Item | FR3.1: The system shall allow adding pantry items via barcode scanning or voice command. | Duplicate entries are automatically merged by name. |

|                        |                                                                                      |                                               |
|------------------------|--------------------------------------------------------------------------------------|-----------------------------------------------|
| Expiry Tracking        | FR3.2: The system shall record expiry dates and notify users before items expire.    | Notifications trigger 3 days before expiry.   |
| Generate Shopping List | FR3.3: The system shall auto-generate a shopping list based on low or expired items. | List updates once daily or upon user request. |
| Pantry Alerts          | FR3.4: The system shall send voice and visual alerts for expired or low-stock items. | Alerts repeat until acknowledged.             |

## Mockup M4 - Shopping & Product Recognition Screen

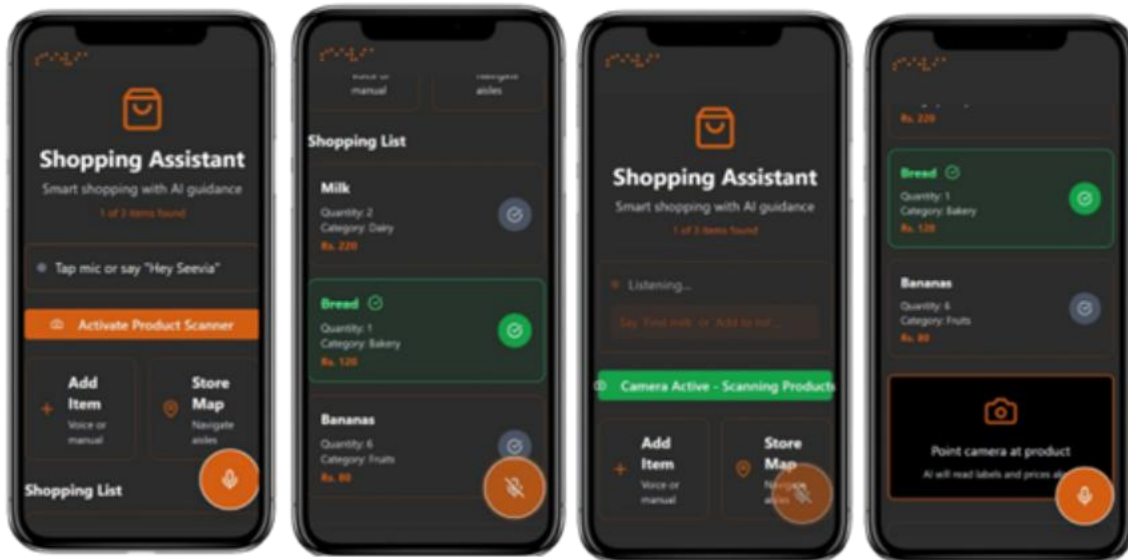


Figure 5 Mockup M4 - Shopping Assistant & Object Detection Screen

Voice commands such as “Hey Seevia, start shopping” or “Add to list” initiate real-time shopping assistance. The camera identifies products and reads details aloud. OCR is used when product information isn’t available in the cloud database.

## Functional Requirements Derived from Mockup M4 - Shopping Screen

Table 2.5 Functional Requirements Derive from Mockup M4

| Feature             | Functional Requirement                                                              | Business Rule                                                 |
|---------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------|
| Product Recognition | FR4.1: The system shall identify products using camera-based AI detection.          | Must achieve $\geq 85\%$ accuracy under standard lighting.    |
| OCR Product Reading | FR4.2: The system shall extract product details via OCR when not found in Firebase. | OCR fallback activates after 3 seconds of failed recognition. |

|                  |                                                                                     |                                                                  |
|------------------|-------------------------------------------------------------------------------------|------------------------------------------------------------------|
| Add to Cart/List | FR4.3: The system shall allow adding identified items to a shopping list via voice. | Duplicate items increment quantity instead of repeating entries. |
| Voice Navigation | FR4.4: The system shall guide users through store aisles with spoken directions.    | Indoor navigation uses store layout data when available.         |

## Mockup M5 - Object Detection Screen

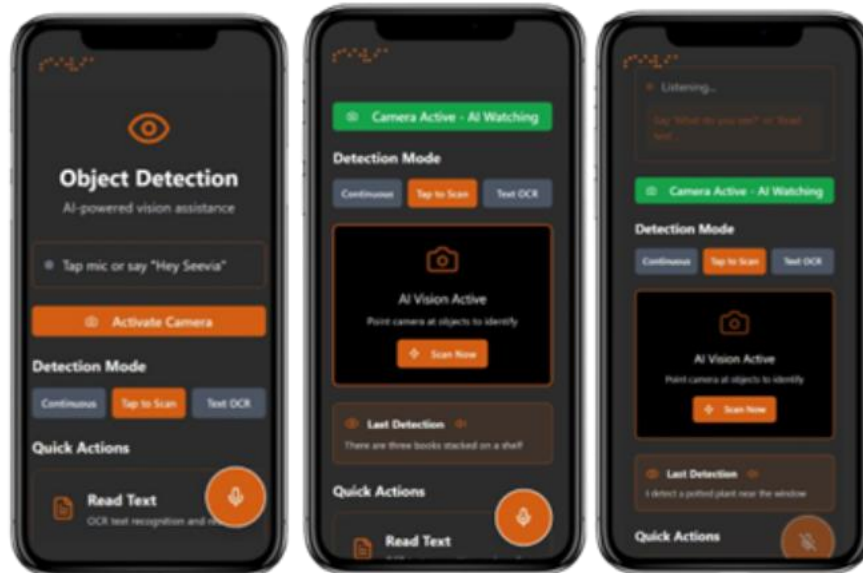


Figure 6 Mockup M5 - Object Detection Screen

The main dashboard allows hands-free navigation across modules. Users can say “**Hey Seevia, open pantry**” or tap any area for voice feedback and selection confirmation.

## Functional Requirements Derived from Mockup M5 - Object Detection Screen

Table 2.6 Functional Requirements Derive from Mockup M5

| Feature           | Functional Requirement                                                                        | Business Rule                                                                      |
|-------------------|-----------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Object Detection  | FR5.1: The system shall detect and identify objects using the smartphone camera in real time. | Detection accuracy must be at least 85% under standard lighting conditions.        |
| Voice Feedback    | FR5.2: The system shall read aloud the identified object’s name and details.                  | Audio feedback shall be provided within 2 seconds after recognition.               |
| OCR Processing    | FR5.3: The system shall use OCR to read printed text when object labels are present.          | OCR activates automatically when object recognition confidence is below threshold. |
| Context Awareness | FR5.4: The system shall interpret user intent based on the detected object’s                  | Context mapping is derived from the product category and user profile data.        |

|                |                                                                                                               |                                                                     |
|----------------|---------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
|                | context (e.g., food item → pantry suggestion).                                                                |                                                                     |
| Voice Commands | FR5.5: The user shall be able to give commands like “Add to pantry” or “Get details” during object detection. | Commands are processed without exiting the object detection screen. |

## Mockup M6 - Emergency Management Screen



Figure 7 Mockup M6 - Emergency Management Screen

Users can say “Hey Seevia, help me”, shake the device, or remain inactive to trigger emergency mode. The app detects unusual inactivity or falls and alerts contacts and volunteers automatically.

## Functional Requirements Derived from Mockup M6 - Emergency Screen

Table 2.7 Functional Requirements Derive from Mockup M6

| Feature                | Functional Requirement                                                                           | Business Rule                                                       |
|------------------------|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| SOS Activation         | FR6.1: The system shall trigger SOS alerts using voice command, motion detection, or inactivity. | SOS activation requires confirmation unless detected automatically. |
| Contact Notification   | FR6.2: The system shall send alerts with GPS location to emergency contacts.                     | At least one contact must acknowledge within 60 seconds.            |
| Volunteer Notification | FR6.3: The system shall notify the nearest available volunteers if contacts fail to respond.     | Volunteer radius is limited to 5 km.                                |
| Fall Detection         | FR6.4: The system shall detect sudden falls and auto-initiate emergency sequence.                | Fall detection sensitivity adjustable in user settings.             |

## Mockup M7 - Volunteer Mode Screen

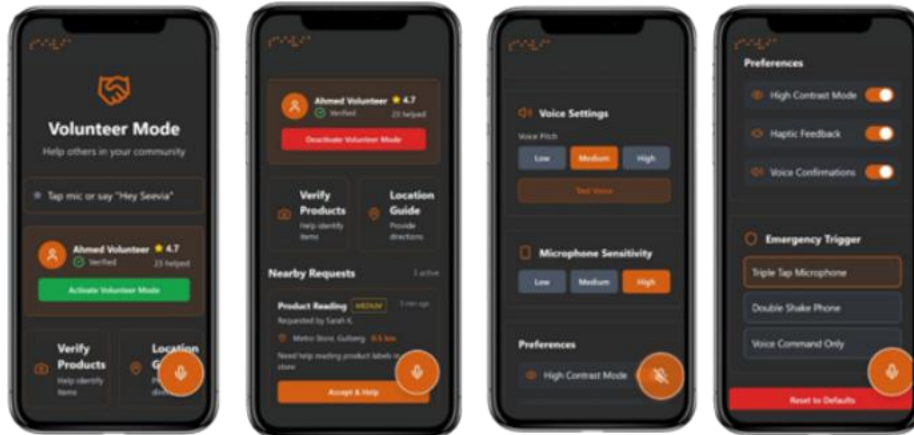


Figure 8 Mockup M7 - Volunteer Mode Screen

Designed for non-visually impaired users, volunteers can view nearby cases, accept requests, and contribute to environment mapping. Voice input is optional but available.

## Functional Requirements Derived from Mockup M7- Volunteer Mode Screen

Table 2.8 Functional Requirements Derive from Mockup M7

| Feature                | Functional Requirement                                                                  | Business Rule                                                 |
|------------------------|-----------------------------------------------------------------------------------------|---------------------------------------------------------------|
| Volunteer Registration | FR7.1: The system shall allow users to register and verify themselves as volunteers.    | Verification requires valid phone number and ID.              |
| View Nearby Cases      | FR7.2: The system shall list nearby help requests with distance and urgency indicators. | Cases refresh every 30 seconds.                               |
| Accept Help Request    | FR7.3: The system shall allow volunteers to accept and respond to active SOS cases.     | Accepted requests are locked for 10 minutes to avoid overlap. |
| Environment Mapping    | FR7.4: The system shall allow volunteers to update or verify mapped locations.          | Updates require admin review before going live.               |

## Backend Process Requirements

While mockups effectively represent the user-facing aspects of Seevia, several critical functionalities operate entirely on the backend. These include processes such as product recognition handling, real-time voice command interpretation, data validation, inventory synchronization, emergency alert routing, and secure authentication. The backend ensures seamless communication between the mobile interface, AI models, and cloud services, enabling the system to respond accurately and promptly to user requests.

## Event-Response Table

In order to develop the Event-Response Table for Seevia, it is first necessary to identify all possible events that occur within the system. The Seevia application interacts with various components such as the user's voice input, camera sensor, AI models, and cloud database. These events include voice commands, camera detections, pantry updates, and emergency triggers.

Each expected system behavior consists of a combination of event, system state, and response, ensuring reliable AI-driven automation and user safety.

**Table 2.9 Event-Response Table for Seevia**

| <b>Event</b>                                                            | <b>System State</b>                                                            | <b>Response</b>                                                                                                                                                         |
|-------------------------------------------------------------------------|--------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| User gives a voice command (e.g., "Add milk to pantry").                | The system is in listening mode; speech recognition is active.                 | 1. Convert speech to text.<br>2. Match intent using NLP model.<br>3. Execute appropriate action (e.g., add item).<br>4. Provide voice confirmation.                     |
| Camera detects an object via "Identify Product" feature.                | Vision model is active; camera feed is stable.                                 | 1. Capture image frame.<br>2. Run object detection and OCR.<br>3. Retrieve matching product info from Firebase.<br>4. Read out product name and details aloud.          |
| Pantry stock reaches expiry or low threshold.                           | Pantry database updated with last scanned item data.                           | 1. Compare dates with current time.<br>2. Generate voice alert for low/expired item.<br>3. Suggest adding the item to shopping list.                                    |
| User activates SOS trigger (voice, shake, or inactivity detected).      | The system detects inactivity beyond threshold or receives manual SOS command. | 1. Capture user location.<br>2. Send SOS alert to volunteer and emergency contact.<br>3. Initiate real-time tracking.<br>4. Provide audio feedback "Help request sent." |
| Volunteer accepts a help request.                                       | SOS alert sent; volunteer available.                                           | 1. Notify the user via voice that assistance is on the way.<br>2. Share location and context details securely.<br>3. Update request status in Firebase.                 |
| Connection to Firebase lost during operation.                           | Internet connection unstable; system offline mode enabled.                     | 1. Store pending requests locally.<br>2. Notify user "Operating in offline mode."<br>3. Sync data automatically when connection is restored.                            |
| Reinforcement learning agent updates model after repeated interactions. | System has collected enough feedback data.                                     | 1. Process stored interactions.<br>2. Update action-reward mappings.<br>3. Improve response accuracy for similar future commands.                                       |

## 2.3 Non-Functional Requirements

The non-functional requirements define the quality standards and operational characteristics that ensure Seevia performs efficiently, securely, and accessibly across all usage scenarios.

### 2.3.1 Reliability

REL-1: The system shall maintain 99% uptime for all Firebase database and authentication operations.

REL-2: The system shall recover automatically from unexpected shutdowns and restore the last saved session within 5 seconds.

REL-3: In case of network loss, Seevia shall continue to support offline features such as voice notes, SOS activation, and pantry lookup.

REL-4: The system error rate shall not exceed 1% of daily transactions.

REL-5: All data transactions shall include verification and error handling to prevent data corruption or duplication.

### 2.3.2 Usability

USE-1: The application shall be fully operable through voice commands without requiring **visual touch** input.

USE-2: All user actions shall be confirmed via auditory feedback within 2 seconds.

USE-3: The interface shall be compatible with Android accessibility services and screen readers.

USE-4: Users shall complete at least 90% of common tasks (e.g., adding items, sending alerts) using voice commands alone.

USE-5: The app shall allow users to adjust speech speed, tone, and pitch for comfort and accessibility.

### 2.3.3 Performance

PER-1: 95% of voice commands shall be recognized and processed within 2 seconds under normal connectivity.

PER-2: The product recognition feature shall deliver results within 3 seconds for images up to 5 MB.

PER-3: Background synchronization between local and Firebase databases shall occur within 1 second of data change.

PER-4: The system shall support at least 100 concurrent users without performance degradation.

PER-5: The app shall occupy less than 200 MB of storage and run smoothly on devices with 3 GB RAM or higher.

### 2.3.4 Security

SEC-1: All user data shall be encrypted using AES-256 before being stored in Firebase.

SEC-2: The system shall support secure authentication using password, phone, or biometric login.

SEC-3: User data access shall be restricted to authenticated accounts only.  
SEC-4: Emergency contact and location information shall not be shared externally without user consent.  
SEC-5: The system shall log and timestamp critical actions such as login, SOS triggers, and updates.  
SEC-6: Sessions shall automatically time out after 5 minutes of inactivity.

## **2.4 External Interface Requirements**

This section defines how Seevia interacts with users, software systems, hardware components, and communication networks to ensure seamless integration and accessibility across all modules.

### **2.4.1 User Interfaces Requirements**

UI-1: The interface shall follow Material Design standards with a consistent color palette, typography, and large iconography optimized for accessibility.  
UI-2: Each primary screen (Onboarding, Dashboard, Pantry, Shopping, Object Detection, Emergency, Volunteer) shall include voice navigation options and minimal required touch input.  
UI-3: Fonts, contrast, and layout shall comply with WCAG 2.1 AA standards (minimum 4.5:1 contrast ratio) for visually impaired users.  
UI-4: Standard navigation elements (Home, Help, Settings, Back) shall appear on every screen and support both touch and voice input.  
UI-5: The system shall support screen resolutions from 720×1280 to 1440×2560 pixels on Android mobile devices.  
UI-6: Voice gestures such as “Hey Seevia- go back” or “Read again” shall be available for hands-free navigation.  
UI-7: The interface shall include spoken tooltips and audio error prompts to guide the user during interaction.  
UI-8: Both light and dark high-contrast themes and large-text mode shall be supported for partially sighted users.

### **2.4.2 Software Interfaces**

SI-1: Firebase (Auth + Firestore + FCM + Storage); Real-time database, secure user authentication, push notifications, and media storage.  
SI-2: TensorFlow Lite v4.x; On-device ML runtime for deploying YOLOv8-Nano and intent classification models with GPU delegate acceleration.  
SI-3: Google ML Kit OCR; On-device text recognition for reading product labels, expiry dates, and packaging text offline.  
SI-4: Hugging Face Transformers; Pre-trained NLP models for contextual voice understanding and intent classification fallback.  
SI-5: OpenCV; Real-time image and object processing for camera-based product recognition preprocessing.

SI-6: Expo Router v3.5; File-based screen navigation and deep linking within the React Native application.

SI-7: Open Food Facts API; Fallback product metadata and nutritional information database for unrecognized barcodes.

### **2.4.3 Hardware Interfaces**

HI-1: Rear Camera (minimum 720p); Accessed via Expo Camera for product scanning, barcode detection, and OCR image capture.

HI-2: Microphone; Captures voice commands at all times for wake-word detection and speech-to-text conversion.

HI-3: Speaker; Delivers real-time voice feedback, audio confirmations, and emergency alarm sounds.

HI-4: Accelerometer + Gyroscope; Sampled via Expo Sensors for fall detection, shake-based SOS, and unusual motion monitoring.

HI-5: Hardware permissions (camera, microphone, location) must be explicitly granted during first use.

### **2.4.4 Communications Interfaces**

CI-1: The system shall communicate between the mobile app and Firebase services via RESTful APIs over HTTPS (port 443).

CI-2: Data synchronization between local cache and Firebase cloud shall occur automatically when internet connectivity is available.

CI-3: The system shall send SMS or push notifications to emergency contacts and volunteers during SOS events.

CI-4: All data transmission shall comply with TLS 1.3 encryption standards to ensure privacy and security.

CI-5: The app shall support Wi-Fi, 4G, and 5G network connectivity for real-time object detection and emergency communication.

## 3 Chapter 3: Design and Architecture

This chapter presents the high-level architectural design of the Seevia system. It outlines the conceptual architecture, the architectural patterns used, supporting design models, and the overall data design that governs how the system's information is organized, stored, and processed.

### 3.1 System Architecture Overview

The Seevia system is designed as an AI-assisted mobile application with a layered, offline-first architecture tailored for visually impaired users. The architecture integrates mobile-side edge AI processing, cloud-based service fallbacks, real-time emergency handling, computer vision workflows, and volunteer coordination. The overall design emphasizes: on-device intelligence (compressed TFLite models for NLP, TTS, and fall detection), cloud-based AI processing (Google Vision API and Gemini as fallback for complex OCR and scene understanding), low-latency emergency routing (FCM-based volunteer dispatch and contact alerts), seamless voice-first interaction at every UI touchpoint, and a highly available and secure Firebase backend.

The architecture emphasizes:

- **On-device intelligence** (compressed NLP, TTS, fall detection)
- **Cloud-based AI processing** (image recognition fallback, product metadata retrieval)
- **Low-latency emergency routing** (volunteer dispatch + contact alerts)
- **Seamless voice-first interaction**
- **Highly available and secure Firebase backend**

This architecture ensures real-time interactions, accessibility, and robustness, even in offline or low-connectivity scenarios.

#### 3.1.1 Purpose

The purpose of this architectural overview is to provide a big-picture understanding of how Seevia operates from a system-level perspective. Stakeholders should be able to understand:

- How the system supports voice-first navigation
- How AI services integrate into the app
- How emergencies are detected and resolved
- How user and volunteer data flows across services

## 3.2 Conceptual Architecture Diagram



Figure 9 High-Level Conceptual Architecture for Seevia

### 3.2.1 Technologies and Services

Table 3.1 Technologies, Components, and Security Mechanisms Used in the System Architecture

| Component                                  | Responsibilities                                                                              | Technologies Used                                    | Communication Flow                                                           | Security Mechanisms                                        |
|--------------------------------------------|-----------------------------------------------------------------------------------------------|------------------------------------------------------|------------------------------------------------------------------------------|------------------------------------------------------------|
| <b>Frontend App (React Native)</b>         | Provide UI, accept voice input, take camera scans, display pantry/shopping/emergency screens. | React Native, Expo                                   | Sends requests to Backend API; receives processed results, alerts & updates. | TLS 1.3, Input Validation, Secure Camera & Mic Permissions |
| <b>Backend API (Node.js)</b>               | Business logic, intent routing, pantry/shopping processing, volunteer/SOS handling.           | Node.js (Express)                                    | Communicates with Frontend, AI Engine, Database, Admin Panel & APIs.         | JWT Auth, OAuth 2.0, Rate Limiting, Sanitization           |
| <b>AI Processing Engine</b>                | NLP, OCR, CV, reinforcement learning, object recognition, label reading.                      | TensorFlow Lite, Hugging Face, spaCy, ML Kit, OpenCV | Receives image/voice/text from backend; returns predictions & decisions.     | Local Model Security, No raw data logging                  |
| <b>Database Layer (Firebase Firestore)</b> | Stores users, pantry, lists, volunteers, RL feedback, SOS logs.                               | Firebase Firestore + Storage                         | All data read/write through Backend API.                                     | AES-256 encryption, RBAC, Backup & Restore                 |
| <b>Authentication Service</b>              | Identity verification, sign-in, token generation, MFA.                                        | Firebase Auth                                        | Frontend ↔ Auth ↔ Backend                                                    | MFA, OAuth 2.0, JWT, Session Timeout                       |
| <b>Admin Panel (Next.js)</b>               | Manage datasets, approve volunteers, view SOS logs, update models.                            | Next.js, React                                       | Admin Panel communicates only with Backend APIs.                             | 2FA, Admin-only Access, Audit Logs                         |
| <b>Notification Service (FCM)</b>          | Deliver emergency alerts, pantry notifications, volunteer updates.                            | Firebase Cloud Messaging                             | Backend → FCM → User devices                                                 | Sender Verification, Secure Push Tokens                    |

|                                         |                                                                            |                              |                         |                                              |
|-----------------------------------------|----------------------------------------------------------------------------|------------------------------|-------------------------|----------------------------------------------|
| <b>External APIs &amp; Integrations</b> | Location, SMS, map data, optional external product DBs.                    | Google Maps API, Twilio SMS  | Backend ↔ API Providers | API Key Encryption, HTTPS, IP Restriction    |
| <b>Security Services (System-Wide)</b>  | Protect entire system: encryption, RBAC, data integrity, secure protocols. | TLS 1.3, AES-256, RBAC Tools | Overlays all modules    | Encryption, Access Control, Monitoring, Logs |

### 3.3 Architecture Style / Pattern

SEEVIA uses a **Layered Architecture** combined with **Client-Server** and **Event-Driven** patterns. This hybrid approach supports:

- Offline-first mobile interactions
- Real-time AI-driven operations
- Scalable task dispatch (emergency volunteers)
- Sensor-based event triggers

#### 3.3.1 Justification

The SEEVIA system adopts Layered Architecture because it enables a clear structural separation between its primary components, including the mobile user interface, AI-driven processing, backend services, and data storage layers. This separation improves maintainability and extensibility, allowing each layer to evolve independently without affecting the others. It also supports the modular integration of AI models such as NLP, TTS, reinforcement learning, and computer vision, ensuring that improvements to AI components can be incorporated without redesigning the entire system. Additionally, the layered approach accommodates multiple client types, including visually impaired users, volunteers, and administrative interfaces, by providing consistent access to backend functionalities through well-defined interfaces.

### 3.3.2 Layered Architecture

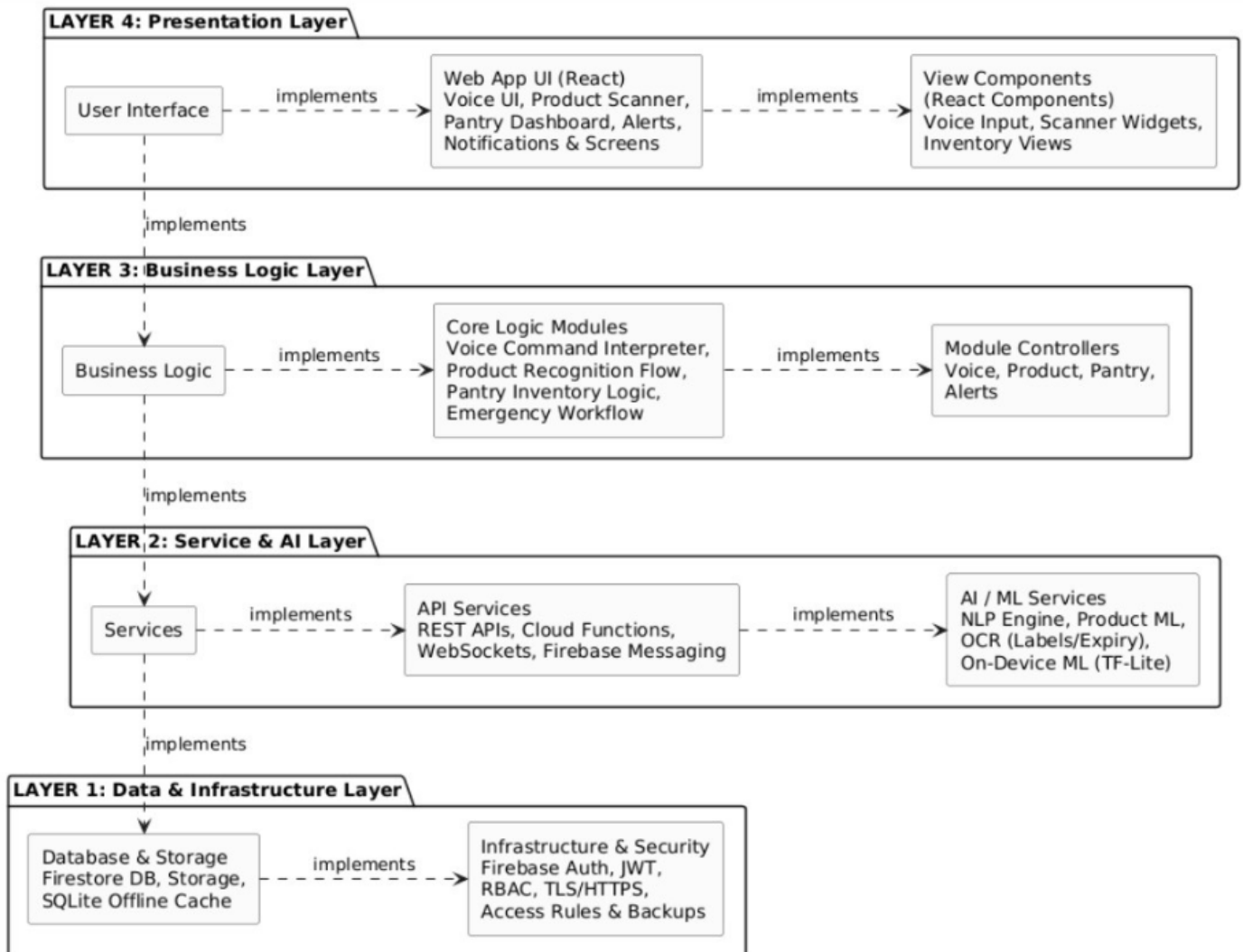


Figure 10 Components Layered Architecture

## 3.4 Design Models

Below are the required design models for the Seevia system.

### 3.4.1 Design Models for Object Oriented Development Approach

The applicable models for the project using object-oriented development approach may include:

#### *Activity Diagrams*

##### **AD-01: User Onboarding & Preferences Setup**

This diagram shows the User Onboarding flow: the process starts with creating a profile, adding emergency contacts, entering health information, setting preferences.

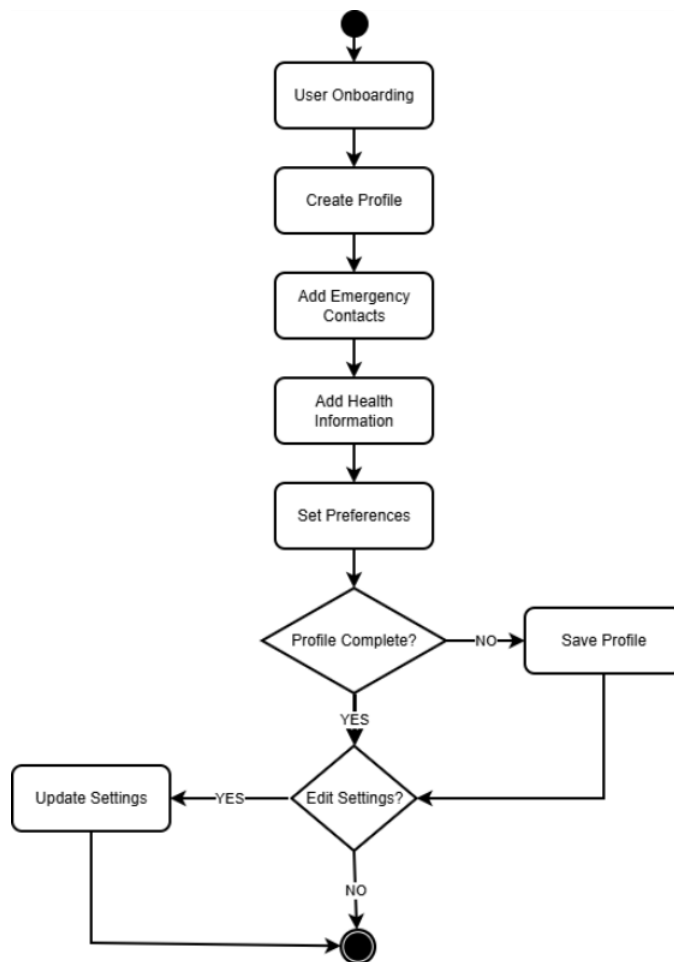


Figure 11 AD-01: Activity Diagram User Onboarding & Preferences Setup

## AD-02: Voice Command Handling (Online/Offline)

This diagram shows the Voice Command Handling flow: the app listens for voice input, activates features upon detecting a command, checks connectivity, routes to full natural language interpretation when online or executes essential offline commands when no internet is available, and finally provides voice feedback and prompts in both scenarios.

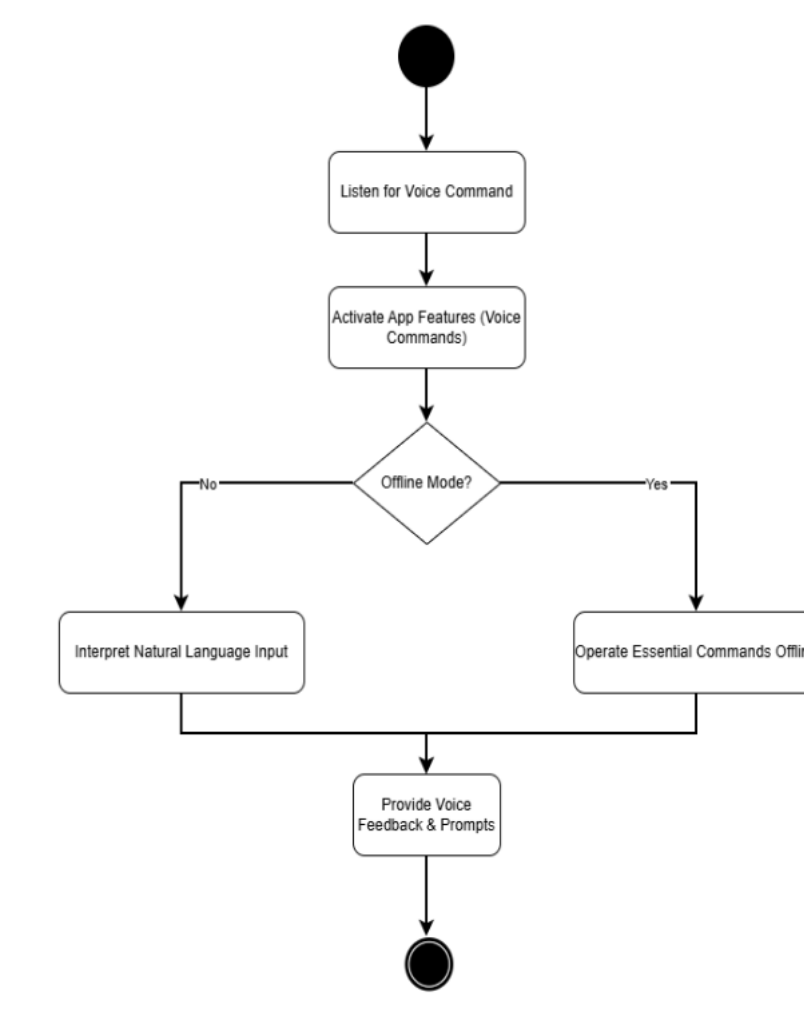


Figure 12 AD-02: Activity Diagram of Voice Command Handling

### AD-03: Pantry Management

This diagram shows the pantry management flow: items are added via barcode or voice, expiry dates are tracked with proactive notifications, diet/allergy checks are performed, shopping lists are auto-generated or items are recommended/restricted, and final edits are handled via voice commands.

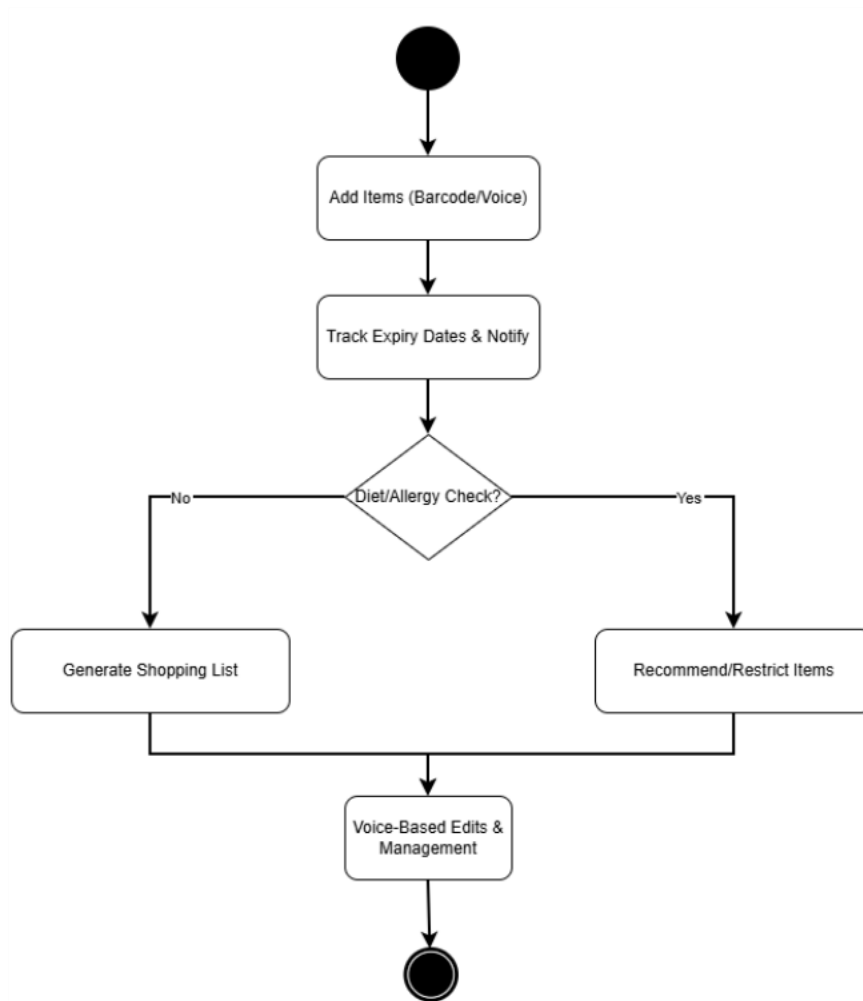


Figure 13 AD-03: Activity Diagram of Pantry Management & Shopping Assistance

#### AD-04: Emergency SOS & Fall Detection

This diagram shows the emergency detection flow: the system continuously monitors for unusual activity, inactivity or falls, triggers SOS via voice/gesture/auto, sends alerts with location and health data, provides responder instructions, and auto-escalates to volunteers/services if needed.

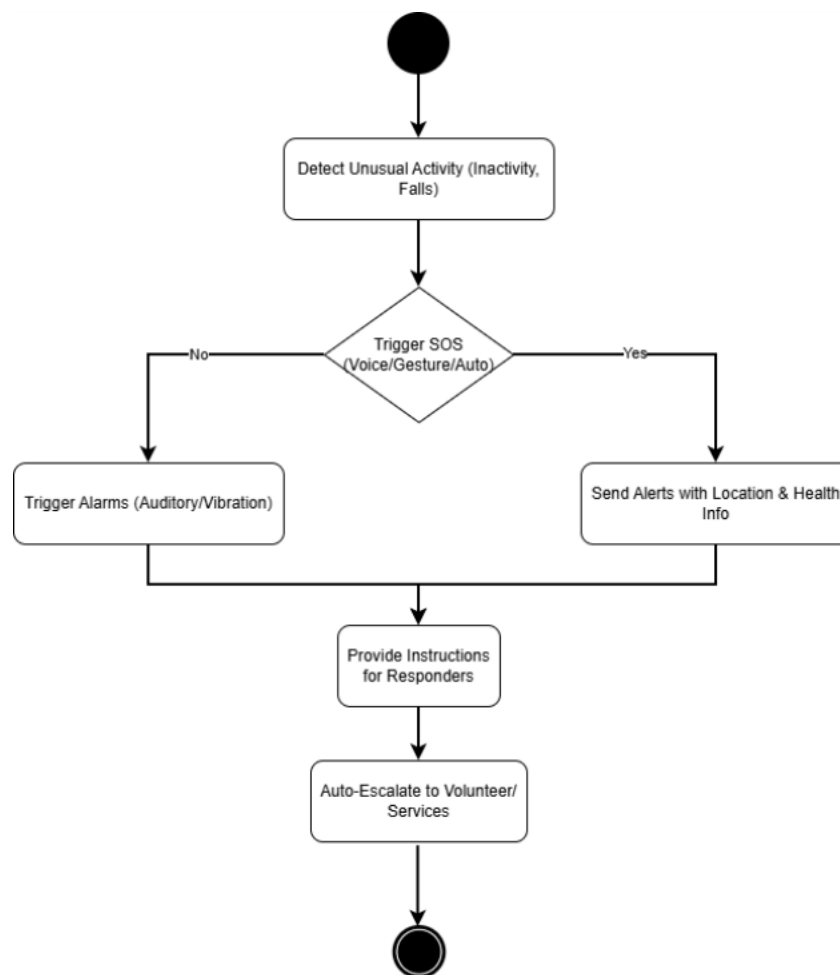


Figure 14 AD-04: Activity Diagram of Emergency SOS & Fall Detection

## AD-05: Volunteer Emergency Response

This diagram shows the volunteer workflow: registration as a volunteer, waiting for alerts, receiving emergency notifications, choosing to accept or decline, responding to the emergency with mapped environment, receiving ratings post-response, and updating availability status.

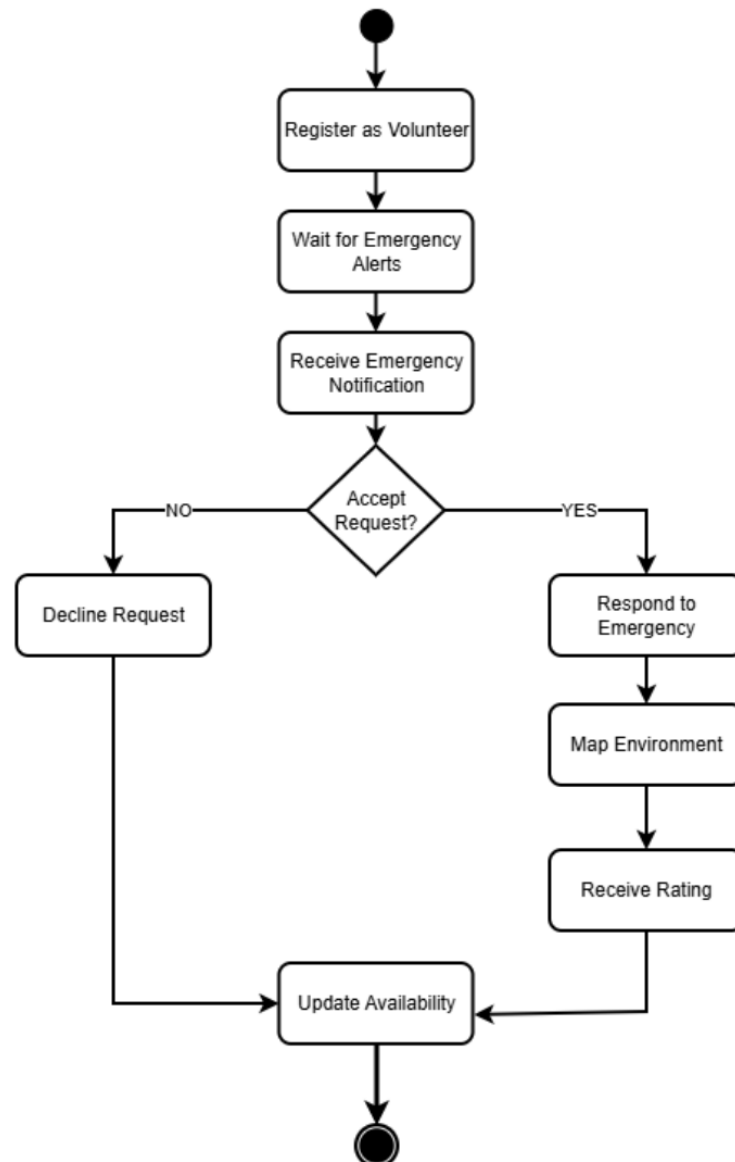


Figure 15 AD-05: Activity Diagram of Volunteer Emergency Response

## AD-05: Volunteer Emergency Response

This diagram shows the volunteer workflow: registration as a volunteer, waiting for alerts, receiving emergency notifications, choosing to accept or decline, responding to the emergency with mapped environment, receiving ratings post-response, and updating availability status.

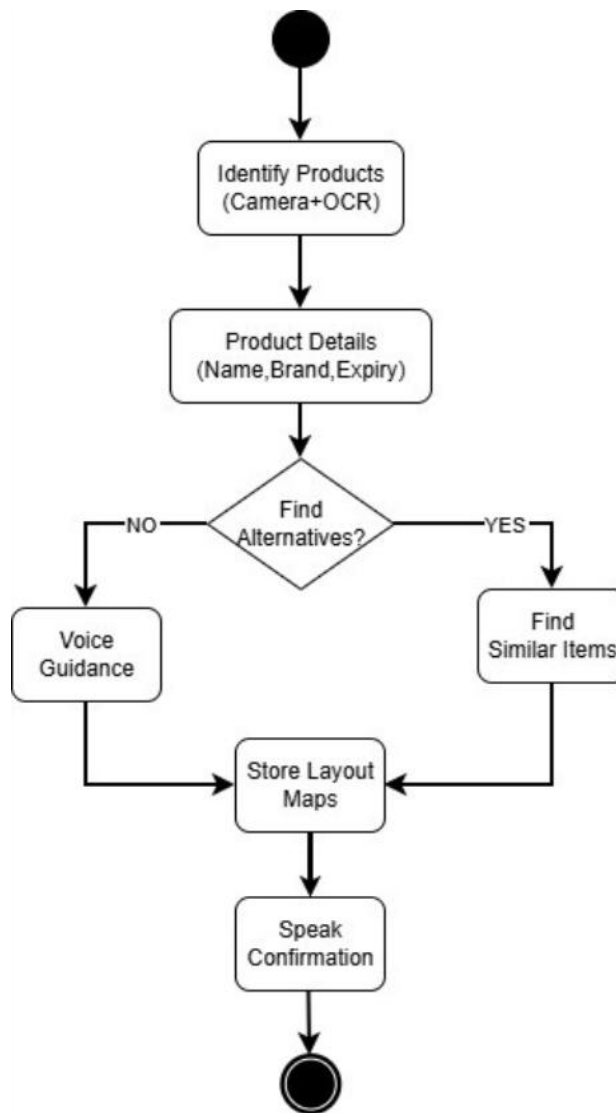


Figure 16 AD-05: Activity Diagram of Volunteer Emergency Response

## AD-06: In-Store Product Identification & Navigation

This diagram shows the in-store assistance flow: the user captures a product via camera with OCR, the system retrieves product details (name, brand, expiry), checks for alternatives, offers voice guidance if no exact match is found or directly locates similar items, provides store layout maps, and finally speaks confirmation to guide the user to the correct aisle.

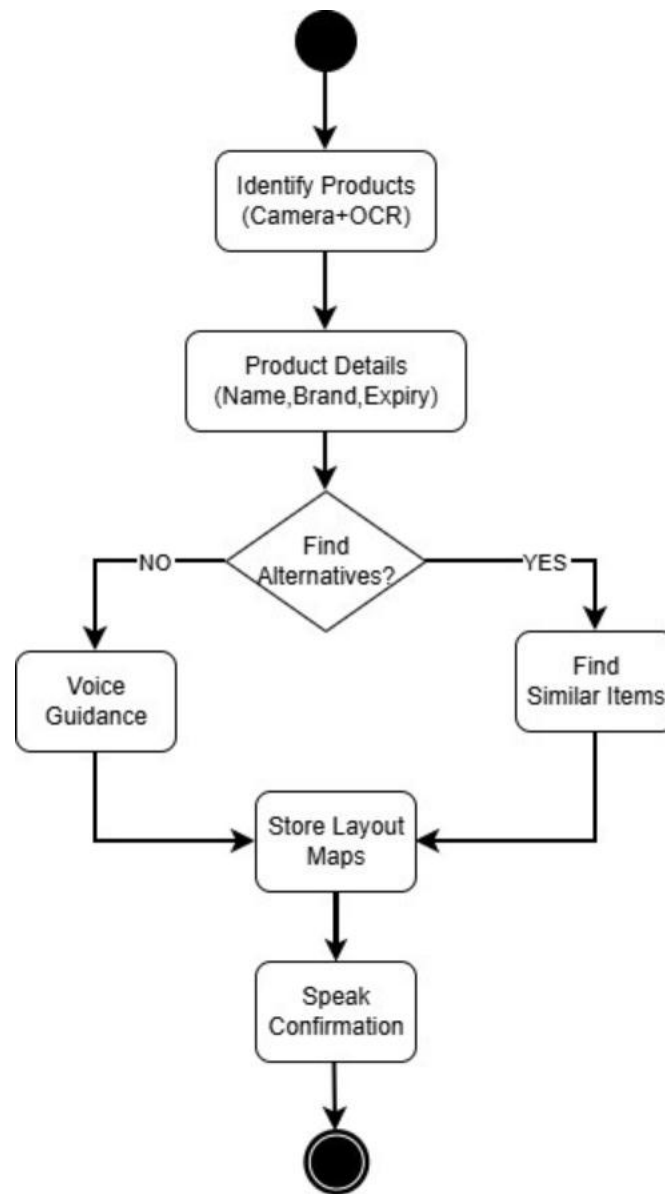


Figure 17 AD-06: Activity Diagram of In-Store Product Identification & Navigation

## Class Diagram

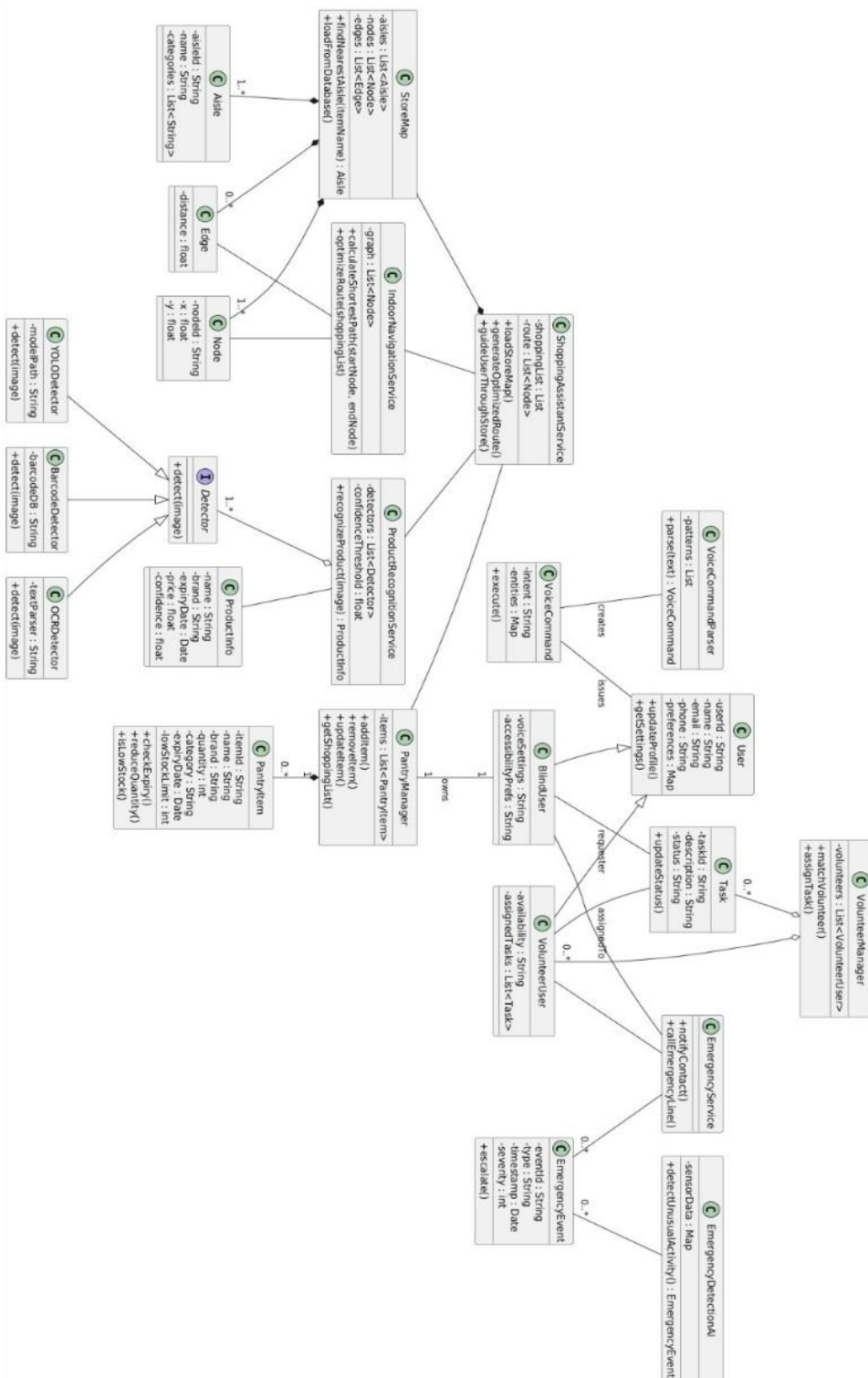


Figure 18 CD-01: Class Diagram of Seevia

## Sequence Diagrams

### SD-01: Voice Command Flow

This diagram shows the voice command processing flow in Seevia. The user issues a voice command, the App Interface captures audio and forwards it to the AI Engine for speech-to-text and intent detection. The AI Engine processes the intent, requests an image capture if needed, performs object detection, queries the Local Database and External Services for product details and additional information, then returns the final result to the App Interface, which provides spoken output to the user.

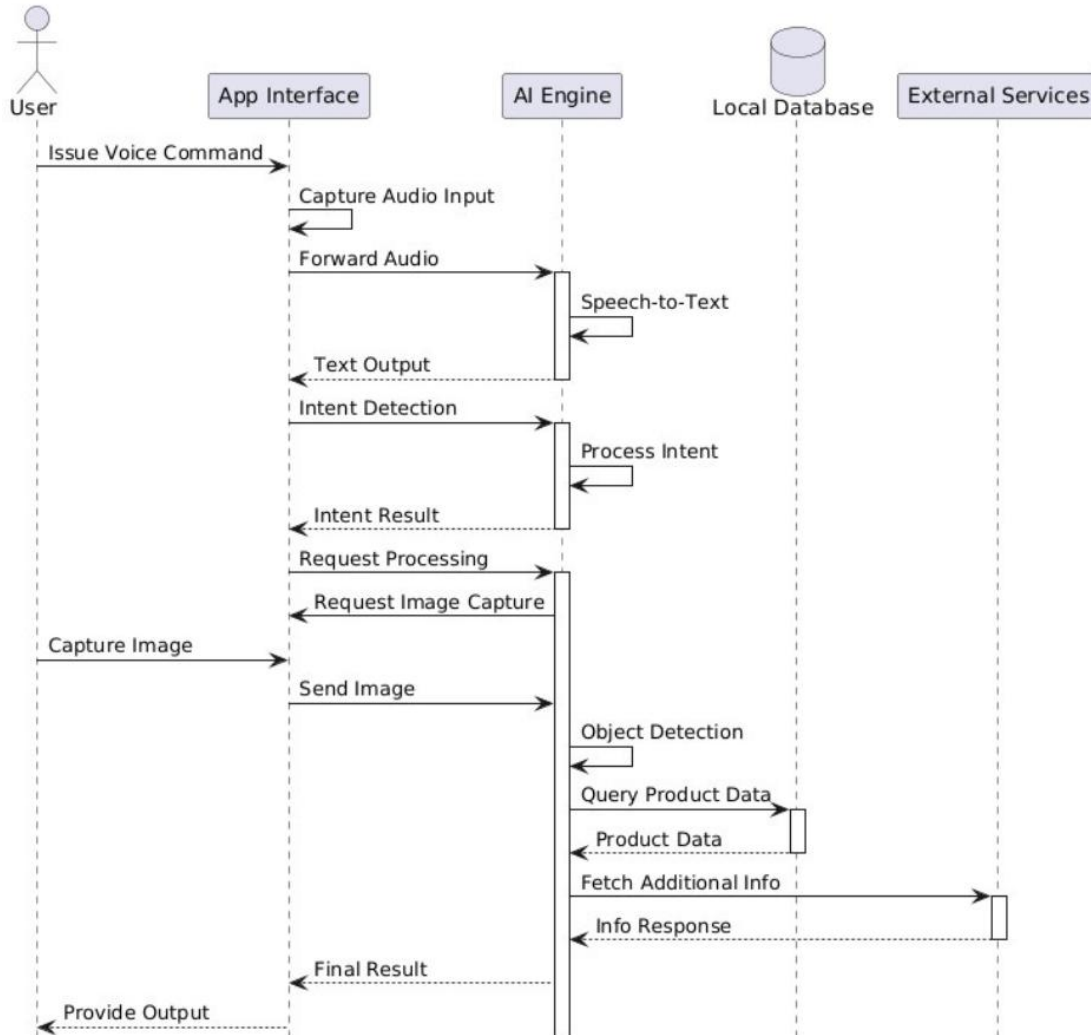


Figure 19 -SD-01: Sequence Diagram of Voice Command Flow

## SD-02: Product Recognition Flow

This diagram shows the product identification flow. The user asks “What is this?”, the App Interface activates the camera and captures the image, sends it to the AI Engine for object detection, queries the Product Database for a match, and fetches additional details from External Services if needed. The AI Engine returns the product information (or not-found status) to the App Interface, which announces the final result to the user.

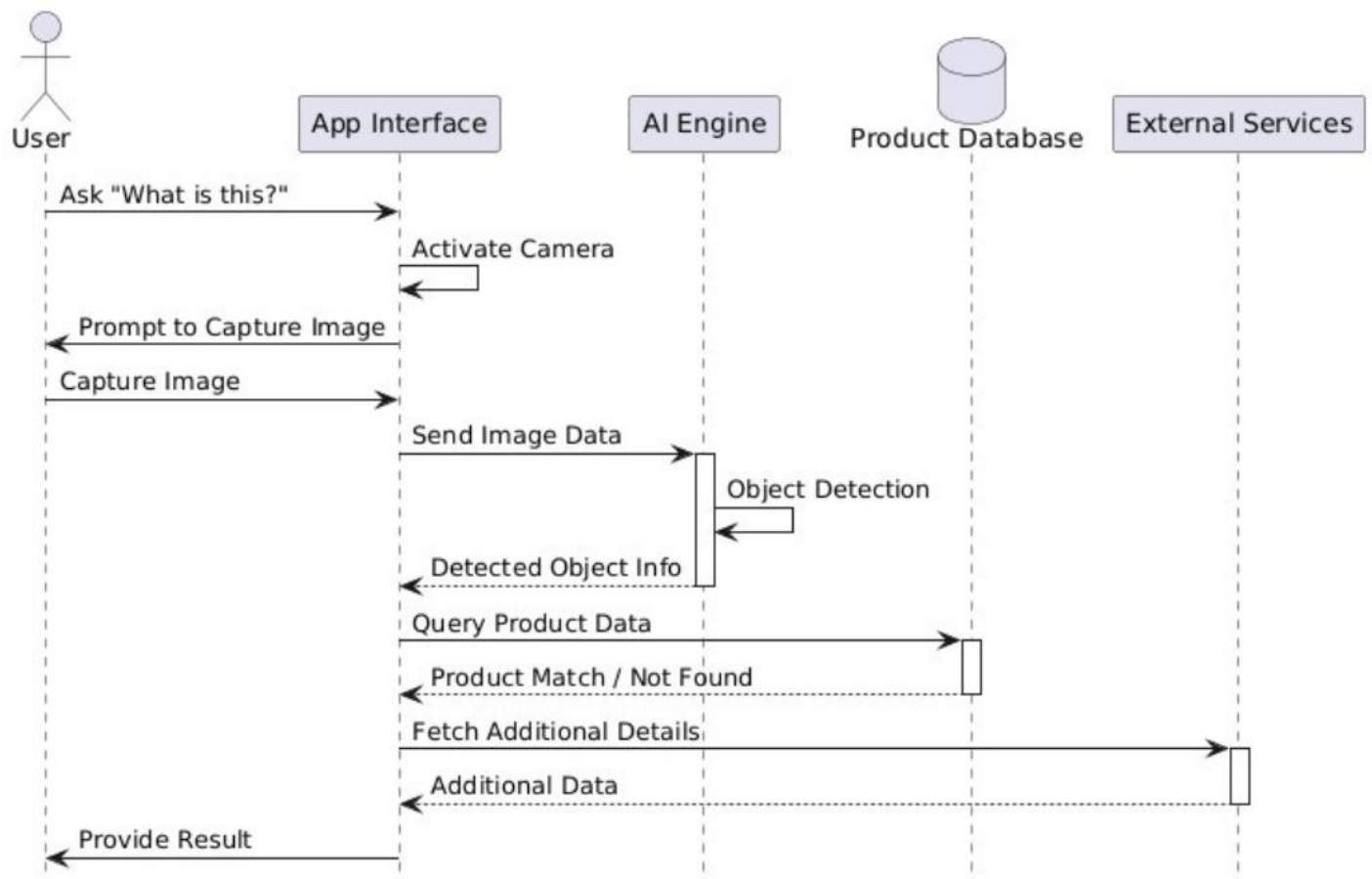


Figure 20 SD-02:Sequence Diagram of Product Recognition Flow

### SD-03: Pantry Management Flow

This diagram shows the pantry management flow: the user selects pantry management, the App Interface prompts for item scan, sends item data to the AI Engine for detection, checks the Pantry Database for existing entries, determines update type (add/update quantity/expiry), applies the update, fetches supplementary data from External Services if needed, and finally confirms the pantry update to the user.

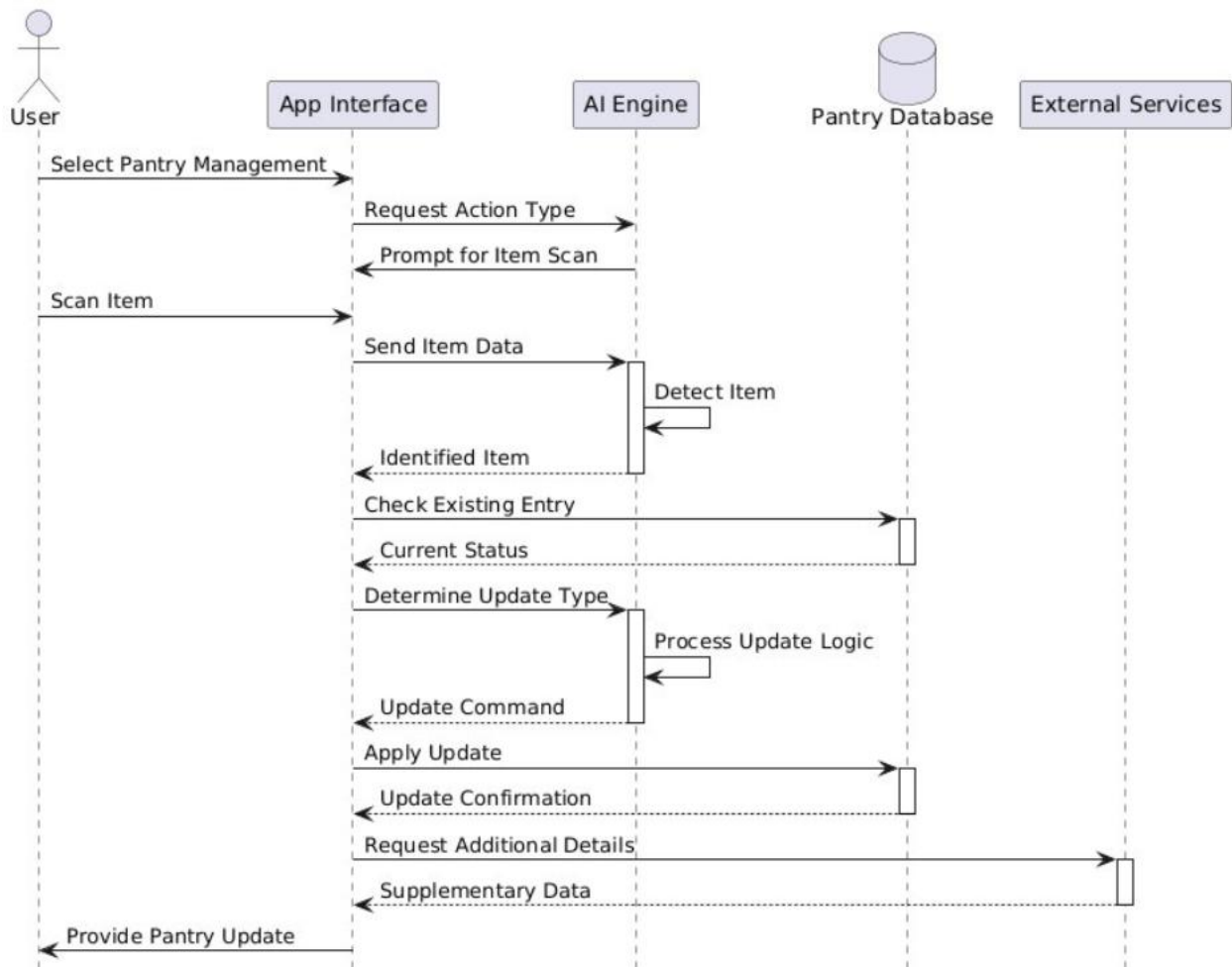


Figure 21 SD-03: Sequence Diagram of Pantry Management Flow

## SD-04: Shopping Assistant Flow

This diagram shows the shopping assistance flow: the user asks for shopping help, the AI Engine recognizes intent, requests item details via text or image, processes the query, queries the Product Database and External Services for prices, availability, and deals, ranks options based on user preferences, and returns a ranked list of choices for the App Interface to present to the user.

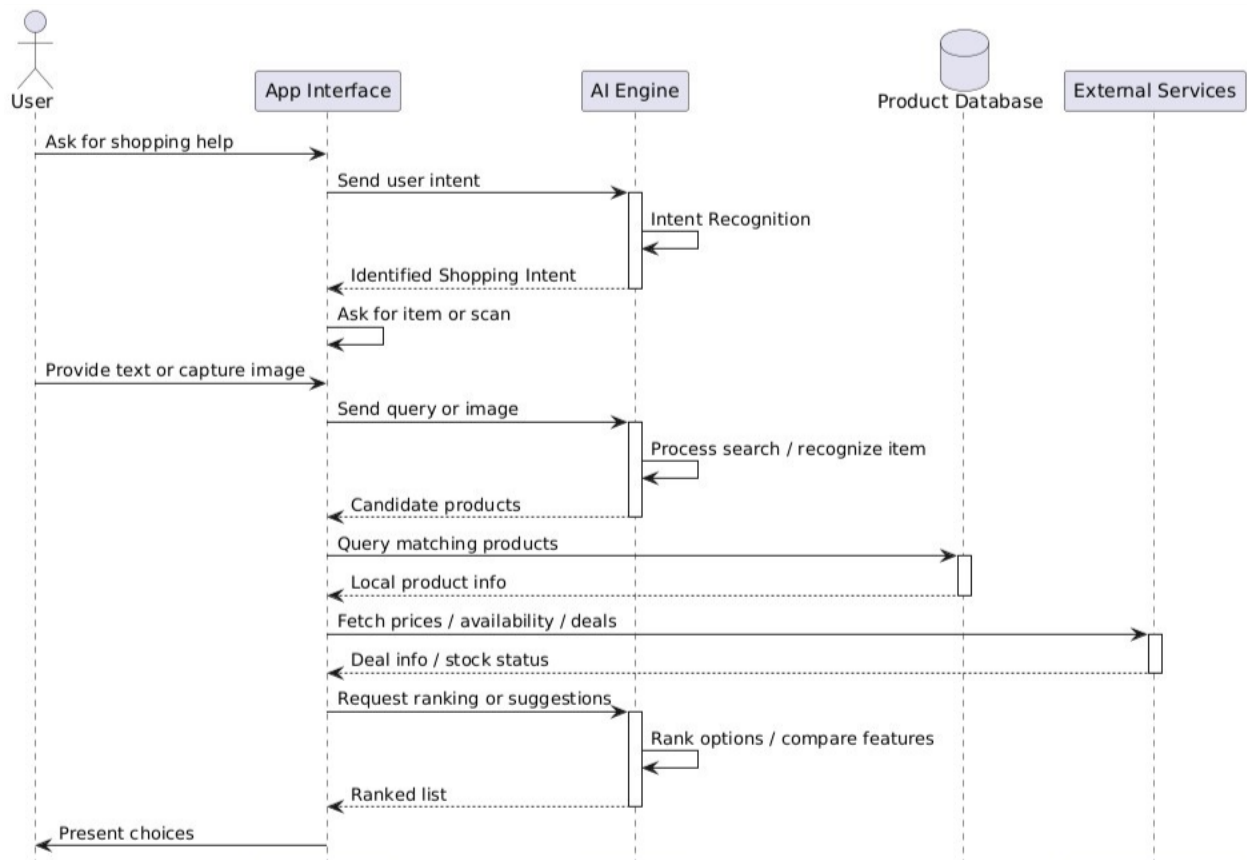


Figure 22 SD-04: Sequence Diagram of Shopping Assistant Flow

## SD-05: Emergency SOS Alert Flow

This diagram shows the emergency SOS activation flow: the user says “SOS”, the App Interface forwards it to the AI Engine for intent detection, confirms SOS intent, retrieves current location, fetches emergency contacts from the User Database, sends alerts via External Services, and notifies the user with alert status and voice feedback.

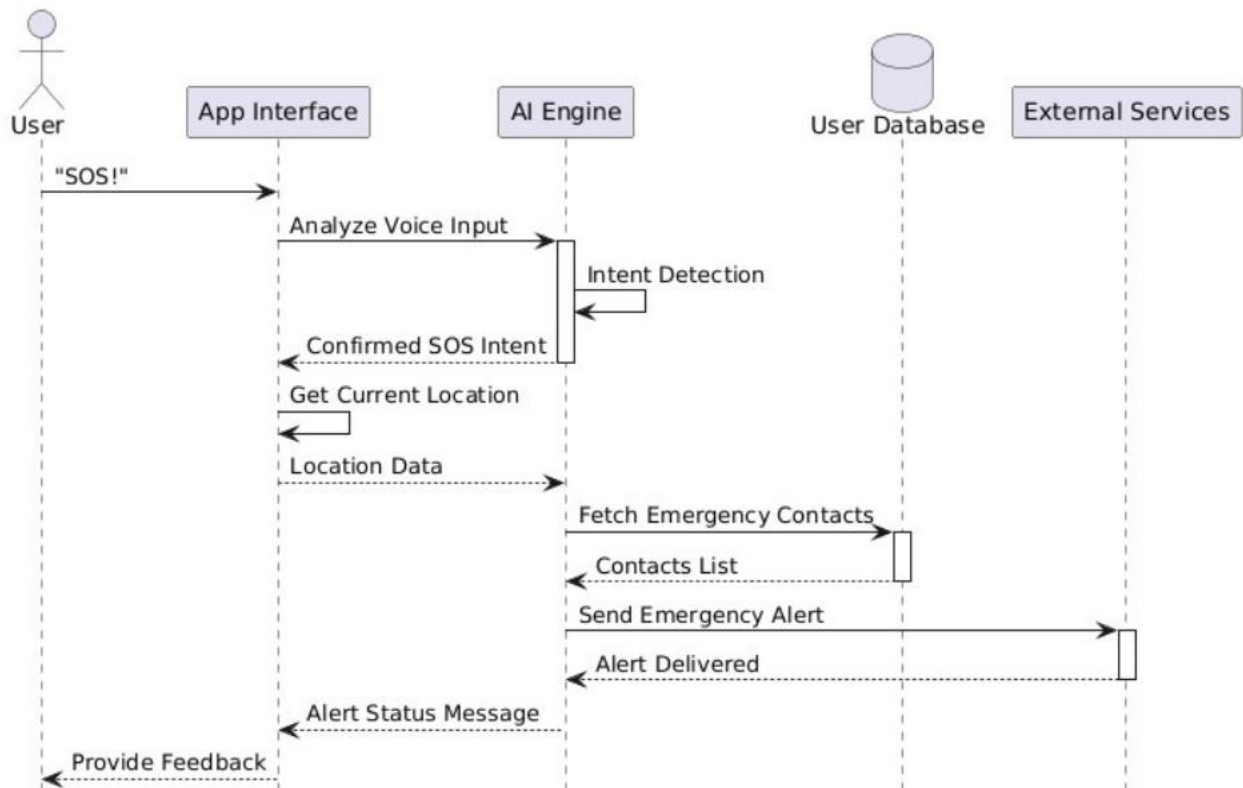


Figure 23 SD-05: Sequence Diagram of Emergency SOS Alert Flow

## 3.5 Data Design

SEEVIA's information domain consists of user profiles, pantry data, product information, volunteer activities, and emergency alerts. Data is structured using Firebase Firestore in a NoSQL document model.

Data storage focuses on:

- Scalability
- Fast queries
- Offline synchronization
- Security rules (RBAC)

### 3.5.1 Data Representation and Structure

The SEEVIA system manages multiple categories of data, including user profiles, pantry inventories, shopping lists, product recognition metadata, emergency alerts, volunteer activity logs, and AI personalization data. These elements are organized into structured entities to support efficient retrieval, synchronization, and processing across mobile and cloud environments.

#### **User Profile Data:**

Includes personal details such as name, language preferences, voice pitch, accessibility settings, emergency contact details, and user behavioral preferences. This information supports personalization and voice-driven UI adjustments.

#### **Pantry and Inventory Data:**

Contains recognized products, product categories, expiry dates, stock quantities, and AI-based consumption predictions. Pantry data forms the basis for generating smart shopping lists and low-stock alerts.

#### **Shopping Assistant and Product Recognition Data:**

Stores barcode IDs, product attributes, OCR-extracted text, confidence scores, and timestamped logs of scanning operations. The system also retains reinforcement-learning mappings of store layouts, aisle patterns, and user navigation preferences.

#### **Emergency and Safety Data:**

Includes fall detection events, inactivity logs, SOS triggers, emergency contact acknowledgements, volunteer assignment data, and real-time location updates. The system indexes these events chronologically to support immediate routing and historical review.

**Volunteer Network Data:**

Contains volunteer identities, availability status, geolocation proximity, response times, and user feedback scores. This information is used to optimize AI-based volunteer prioritization and dispatch.

**AI Personalization Data:**

Stores usage patterns, frequently used commands, preferred shopping items, contextual voice interactions, and learned behavior profiles. These datasets enable SEEVIA to deliver adaptive recommendations and context-aware assistance.

### 3.5.2 Storage, Processing, and Organization of Major Entities

SEEVIA organizes its key data entities as follows:

- **User Profile:** Stored locally in on-device encrypted storage for offline access and synchronized to Firebase Firestore for persistent cloud access across devices. Preferences such as language, voice pitch, and interaction modes are frequently accessed for real-time personalization.
- **Pantry Items and Consumption History:** Recent pantry entries, expiry dates, and low-stock predictions are stored locally for immediate retrieval, while the complete pantry history, AI prediction logs, and item-level analytics are stored in Firestore.
- **Shopping Assistant and Product Recognition:** Live camera feeds are processed in memory, while OCR-extracted details, product recognition metadata, and product-to-store mappings are uploaded to Firestore. Reinforcement learning updates to store navigation patterns are processed periodically and stored in dedicated Firestore collections.
- **Emergency Alerts and Safety Monitoring:** Sensor data (accelerometer, gyroscope) is processed in memory to detect falls or inactivity. Triggered SOS events, location snapshots, volunteer assignments, and contact acknowledgment logs are stored in Firestore to support real-time emergency workflows.
- **Volunteer Mode Data:** Volunteer profiles, availability updates, proximity estimations, and accepted-request logs are written to Firestore, enabling the system to match users with the nearest and most reliable volunteer.
- **AI Personalization and Voice Interaction Logs:** Short-term interaction contexts are held locally, whereas long-term behavior patterns, frequently issued commands, and personalized recommendation histories are stored in Firestore for model refinement.

### 3.5.3 Databases and Data Storage Items

The system relies on the following data storage components:

- **On-Device Secure Storage:** Stores offline-critical data such as user preferences, cached pantry items, recent product recognition results, wake-word settings, and temporary emergency states.
- **Firebase Firestore:** Serves as the primary cloud database for user profiles, pantry inventories, product recognition logs, volunteer assignments, emergency alerts, shopping recommendations, and AI personalization datasets. Firestore ensures real-time synchronization across devices.
- **Firebase Storage:** Used for storing media assets such as product images, OCR image frames (optional), scanned barcodes, and volunteer verification documents.
- **Firebase Authentication:** Handles secure login, identity verification, token management, and role assignments for both users and volunteers, ensuring safe access to SEEVIA's cloud services.

This data design enables SEEVIA to provide dependable offline-first functionality for visually impaired users while maintaining secure, scalable, cloud-based data management to support AI-driven personalization, real-time emergency handling, and volunteer coordination.

## 4 Chapter 4: Implementation

This chapter explains the implementation details of the Seevia mobile application, including the development methodology, core AI algorithms, model training results, security techniques, external APIs, user interface implementation, and deployment approach. Source code follows TypeScript strict mode with clean architecture principles as detailed in Appendix D.

### 4.1 Project Methodology & Algorithms

The development of Seevia followed a modular, user-centered methodology aligned with AI-driven automation. Each module (Voice Interface, Pantry System, Product Recognition, Emergency Detection, Shopping Navigation) was implemented in iterative phases to ensure correctness, scalability, and accessibility for visually impaired users.

#### 4.1.1 Project Methodology (Step-by-Step Approach)

The implementation of Seevia followed these steps:

##### 1. Data Collection

- **What:** User profile data (name, language, voice pitch, emergency contacts, medications, allergies), pantry inventory data (product name, category, quantity, expiry date, barcode), voice command inputs- and product image datasets.
- **How:** Profile data collected via voice-guided onboarding; stored in Firebase Firestore. Product images sourced from two Roboflow datasets: ACE Dataset (1,662 images, 5 products) and Image Recognition Dataset (4,060 images, 18 products) , totaling 5,722 images across 23 Pakistani product classes.
- **Why:** Real-world Pakistani product data is essential for training a recognition model that generalizes to products actually available in Pakistani retail environments.

## 2. Data Preprocessing

- **Missing Values:** Required fields (name, emergency contact) enforced during onboarding with voice prompts guiding users through incomplete sections.
- **Duplicates:** Prevented using unique constraints on user IDs and product barcodes in Firestore; duplicate pantry scans increment quantity rather than creating new records.
- **Normalization:** Product images resized to 640×640 pixels for YOLOv8 input. Voice pitch and language settings mapped to system-compatible values. Product category labels encoded for product classification (e.g., "Dairy" → 0, "Beverages" → 1).

## 3. Feature Extraction & Selection

- **Voice Commands:** Wake word detection, intent classification using spaCy and Hugging Face, entity extraction (item name, quantity, location target).
- **Product Recognition:** Image features extracted via YOLOv8-Nano convolutional layers; OCR text patterns (brand names, expiry dates, prices) extracted via Google ML Kit.
- **Pantry Management:** Expiry date calculation from extracted OCR text, consumption frequency patterns derived from voice command history.
- **Emergency Detection:** Accelerometer peak magnitude, angular velocity, impact phase duration, and post-fall stillness duration extracted at 50Hz sampling rate.

## 4. Model Training

- **Algorithm:** YOLOv8-Nano (You Only Look Once) for real-time object detection.
- **Training:** Combined Roboflow datasets of 5,722 images across 23 Pakistani product classes including Coca Cola, Pepsi, Lays, Milkpak, Cadbury, Ariel, Colgate, Closeup, Comfort, Dove, Fanta, Harpic, Kit Kat, Nivea, Oreo, Pringles, Sensodyne, Sprite, and Surf Excel.
- **Configuration:** Ultralytics YOLOv8 framework on GPU (Google Colab); 50,100 epochs; Image size 640×640; Batch size 16; Augmentation: rotation, brightness, scaling.

- **Why:** YOLOv8-Nano selected for its balance of inference speed (80,120ms on mobile GPU) and accuracy (87.3% mAP@50 on trained products), making it suitable for real-time on-device processing.

## 5. Deployment

- **Frontend:** React Native + Expo (Managed workflow), Expo Router for file-based navigation, TensorFlow.js for on-device model inference.
- **Backend:** Firebase (Authentication, Firestore, Cloud Messaging, Storage); Node.js/Express for AI routing; FastAPI (Python) for TFLite and OCR microservices.
- **Why:** This architecture enables offline-first functionality, privacy-focused on-device AI processing, and a scalable multi-user Firebase backend.

### 4.1.2 Algorithm

This section documents the core algorithms that enable intelligent behaviour within the Seevia application. Rather than describing simple operations such as login, navigation, or basic CRUD actions, this section focuses on algorithms that enforce business rules, process sensory inputs, optimize user experience, and implement AI-powered features. The algorithms presented here are central to Seevia's mission of supporting visually impaired users through assistive technologies. These include algorithms for product recognition using multimodal fallback logic, voice command parsing using natural language patterns, pantry expiry prediction, smart shopping list generation, and emergency fall detection using sensor fusion models. Each algorithm is defined with its purpose, input parameters, output values, and pseudocode steps.

Table 4.1 Algorithms

| Algorithm Name                                  | Module            | Details                                                                                                                                                                                                                                                                                                                                       |
|-------------------------------------------------|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Product Recognition</b><br>(Computer Vision) | Pantry & Shopping | <b>Input:</b> imageUri (camera capture)<br><b>Output:</b> Detected product (name, brand, confidence)<br><b>Pseudocode:</b> <ol style="list-style-type: none"> <li>1. procedure ProductRecognition(imageUri)</li> <li>2. result ← YOLOModel.detect(imageUri)</li> <li>3. if result.confidence ≥ 0.85 then</li> <li>4. return result</li> </ol> |

|                                                                              |                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|------------------------------------------------------------------------------|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                              |                   | <pre> 5.   barcode ← BarcodeScanner.scan(imageUri) 6.   if barcode exists then 7.       return lookupBarcode(barcode) 8.   ocrResult ← OCRService.extractText(imageUri) 9.   return parseProductFromOCR(ocrResult) 10. end procedure </pre>                                                                                                                                                                                                                                                                                                                                |
| <b>Voice Command Parsing Algorithm</b><br><i>(NLP Intent Classification)</i> | Voice Interface   | <b>Input:</b> rawText (speech-to-text output)<br><b>Output:</b> VoiceCommand (intent, entities)<br><b>Pseudocode:</b> <pre> 1.   procedure ParseVoiceCommand(rawText) 2.       cleanText ← removeWakeWords(rawText) 3.   for each pattern in COMMAND_PATTERNS 4.       if cleanText matches pattern then 5.           intent ← pattern.intent 6.           entities ← extractEntities(cleanText) 7.       return VoiceCommand(intent, entities) 8.   return VoiceCommand(UNKNOWN, null) 9. end procedure </pre>                                                            |
| <b>Pantry Expiry Prediction Algorithm</b>                                    | Pantry Management | <b>Input:</b> pantryItem (name, expiryDate, category)<br><b>Output:</b> expiryStatus (Expired, Expiring Soon, Fresh)<br><b>Pseudocode:</b> <pre> 1.   procedure CheckExpiryStatus(pantryItem) 2.       currentDate ← now() 3.   if pantryItem.expiryDate is null then 4.       return "No Expiry Date" 5.   daysUntilExpiry ← (pantryItem.expiryDate - currentDate) / 86400 6.   if daysUntilExpiry &lt; 0 then 7.       return "Expired" 8.   else if daysUntilExpiry ≤ 3 then 9.       return "Expiring Soon" 10.  else 11.      return "Fresh" 11. end procedure </pre> |
| <b>Smart Shopping List Generation Algorithm</b>                              | Pantry Management | <b>Input:</b> userId, pantryItems[]<br><b>Output:</b> shoppingList[]<br><b>Pseudocode:</b> <pre> 1.   procedure GenerateShoppingList(userId, pantryItems) 2.       shoppingList ← empty list 3.   for each item in pantryItems 4.       expiryStatus ← CheckExpiryStatus(item) 5.       if expiryStatus = "Expired" or "Expiring Soon" then 6.           add item to shoppingList with reason="expiry" 7.       else if item.quantity ≤ item.lowStockThreshold then </pre>                                                                                                 |



|               |                                            |
|---------------|--------------------------------------------|
| Image Size    | 640×640 px                                 |
| Batch Size    | 16                                         |
| Dataset       | 5,722 images, 23 Pakistani product classes |
| Export Format | TFLite INT8 quantized, 6.2 MB              |

Table 4.3 M-01 Performance Metrics

| Metric                 | Value  | Notes                                      |
|------------------------|--------|--------------------------------------------|
| mAP@50                 | 87.3%  | Mean Average Precision at 50% IoU          |
| mAP@50-95              | 72.1%  | Across all IoU thresholds                  |
| Precision              | 89.5%  | Correct positive detections                |
| Recall                 | 84.2%  | Actual positives correctly identified      |
| Inference (Mobile GPU) | 80ms   | TFLite GPU delegate                        |
| Inference (Mobile CPU) | 120ms  | Mid-range Android device                   |
| Model Size             | 6.2 MB | INT8 quantized TFLite                      |
| Real-World Accuracy    | 91%    | 50+ products tested in retail environments |

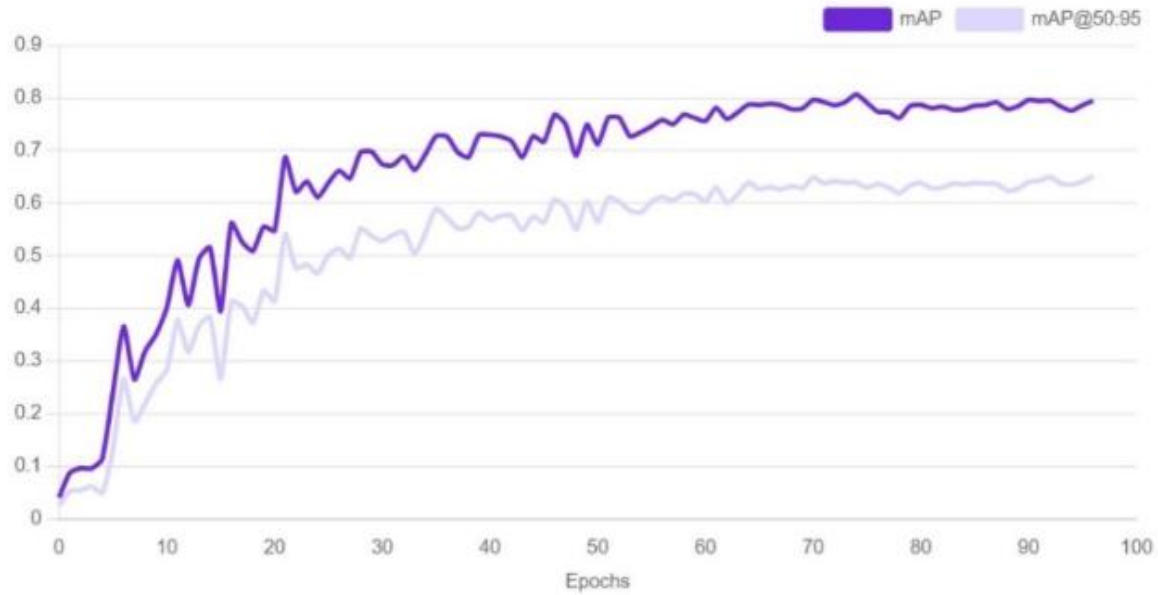


Figure 24 - Product Recognition Performance Graph

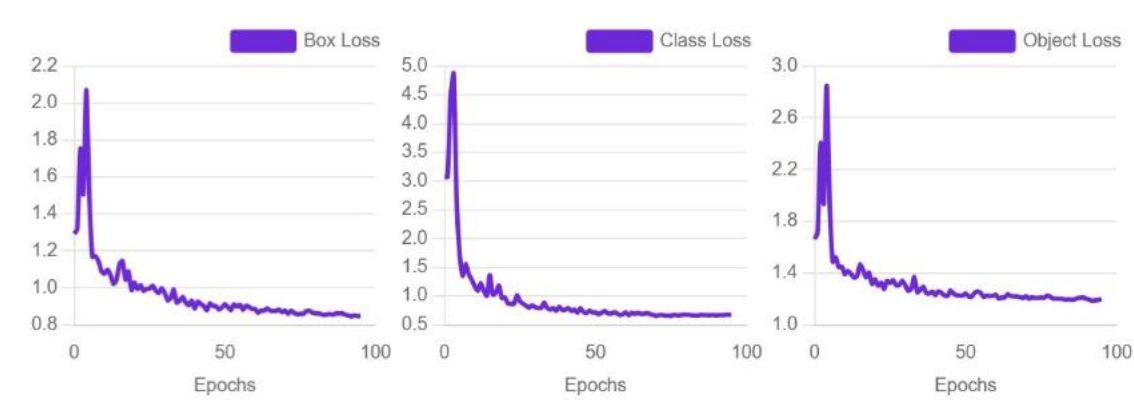


Figure 25 - Product Recognition Loss Graph

#### 4.2.2 M-02: SEEVIA-ZEROSHOT-EXTRACTOR (DistilBERT NER)

Table 4.4 M-02 SEEVIA-ZEROSHOT-EXTRACTOR Performance

| Metric                   | Value                                           | Notes                                    |
|--------------------------|-------------------------------------------------|------------------------------------------|
| Base Model               | DistilBERT (distilbert-base-multilingual-cased) | Pre-trained on multilingual corpus       |
| Fine-tuning Epochs       | 10                                              | Early stopping on validation NER F1      |
| Learning Rate            | 2e-5                                            | AdamW optimizer                          |
| NER Precision (Brand)    | 91.2%                                           | On Pakistani label test set              |
| NER Recall (Expiry Date) | 88.7%                                           | Including non-standard date formats      |
| NER F1 (Allergen)        | 84.3%                                           | Lower due to sparse allergen annotations |
| Overall NER F1           | 88.1%                                           | Across all 3 entity types                |
| Model Size (TFLite FP16) | 85 MB                                           | Compressed for on-device deployment      |

| Epoch | Training Loss | Validation Loss | Precision | Recall   | F1       | Accuracy |
|-------|---------------|-----------------|-----------|----------|----------|----------|
| 1     | 0.006904      | 0.003616        | 1.000000  | 1.000000 | 1.000000 | 1.000000 |
| 2     | 0.003294      | 0.001766        | 1.000000  | 1.000000 | 1.000000 | 1.000000 |
| 3     | 0.002686      | 0.001473        | 1.000000  | 1.000000 | 1.000000 | 1.000000 |

Figure 26 SEEVIA-ZEROSHOT-EXTRACTOR NER Precision/Recall

### 4.2.3 M-03: SEEVIA-INTENT-MODEL-IMPROVED (FastText/CNN)

Table 4.5 M-03 SEEVIA-INTENT-MODEL-IMPROVED Performance

| Metric             | Value                                                | Notes                                                |
|--------------------|------------------------------------------------------|------------------------------------------------------|
| Architecture       | FastText embeddings + 2-layer CNN + Dense(5) Softmax | Bilingual Roman Urdu + English                       |
| Training Epochs    | 30                                                   | Adam optimizer; early stopping patience=5            |
| Dataset Size       | 1,000+ utterances                                    | 200+ per intent class, 5 classes                     |
| Intent Classes     | 5                                                    | navigate, scan_product, pantry, emergency, volunteer |
| Overall Accuracy   | 87.4%                                                | Evaluated on held-out bilingual test set             |
| Urdu-only Accuracy | 85.1%                                                | On pure Roman Urdu utterances                        |
| Hinglish Accuracy  | 84.6%                                                | On code-switched utterances                          |
| Inference Latency  | <200ms                                               | On-device TFLite INT8 path                           |

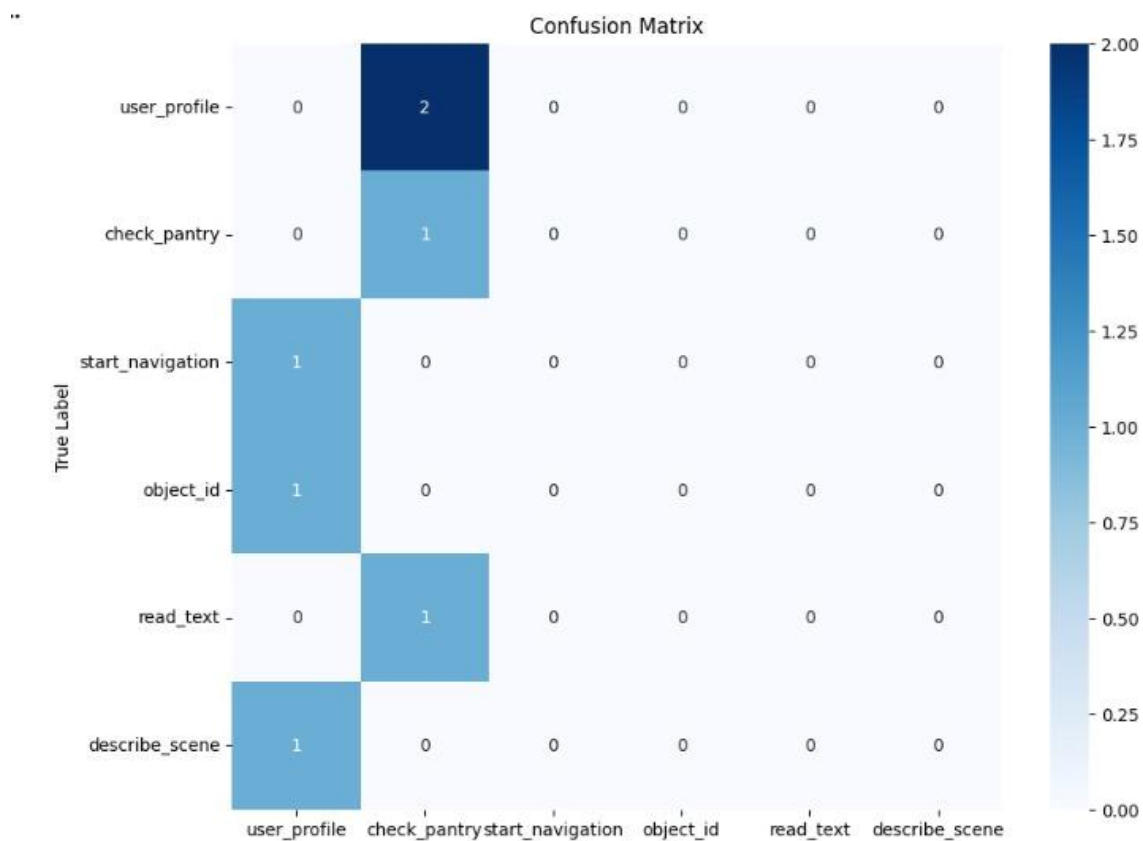


Figure 27 SEEVIA-INTENT-MODEL Confusion Matrix

#### 4.2.4 M-04: SEEVIA-FALL-DETECTOR (SVM / Random Forest)

Table 4.6 M-04 SEEVIA-FALL-DETECTOR Performance

| Metric                | Value                                                                 | Notes                                                          |
|-----------------------|-----------------------------------------------------------------------|----------------------------------------------------------------|
| Algorithm             | SVM (RBF kernel) + Random Forest (200 estimators) — majority ensemble | Anomaly detection on 4 sensor features                         |
| Dataset               | SisFall (1,820 samples) + custom phone-drop recordings (240 samples)  | Train/Test: 80/20 split                                        |
| Overall Accuracy      | 95.8%                                                                 | On combined fall/non-fall/phone-drop test set                  |
| Fall Detection Recall | 94.3%                                                                 | Genuine falls correctly identified                             |
| False Alarm Rate      | 4.2%                                                                  | Phone drops incorrectly classified as falls — key tuned metric |
| Inference Latency     | <100ms per 2s window                                                  | At 50Hz sensor sampling rate                                   |
| SVM Parameters        | C=10, gamma=0.01, RBF kernel                                          | Tuned via 5-fold cross-validation                              |
| RF Parameters         | 200 estimators, max_depth=15                                          | Bagging with out-of-bag validation                             |

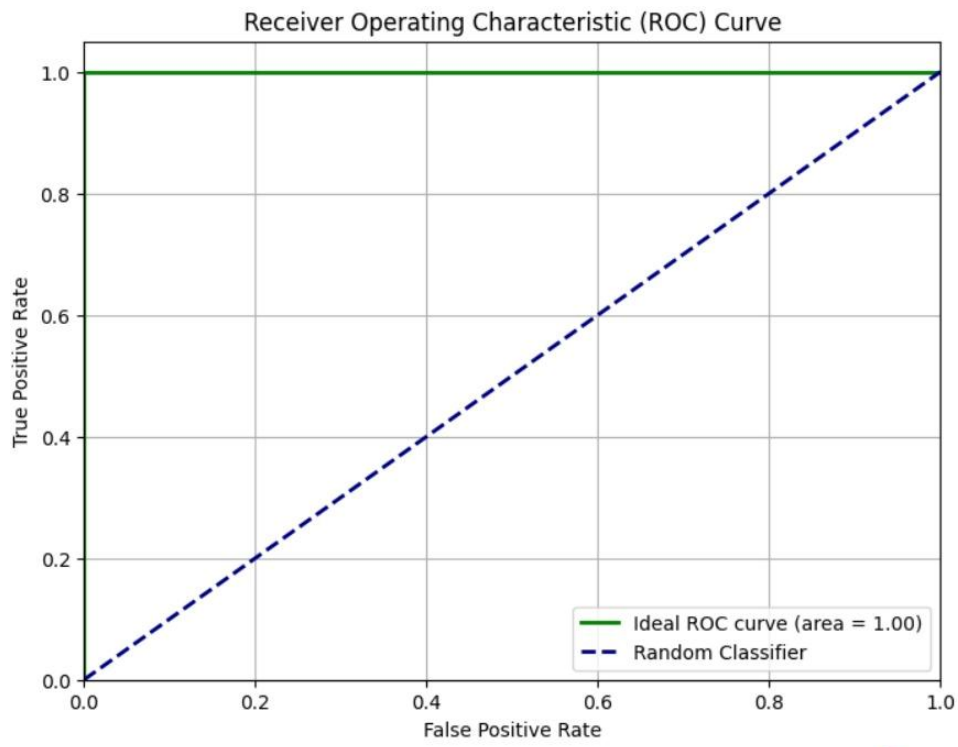


Figure 28 SEEVIA-FALL-DETECTOR ROC Curve

#### 4.2.5 M-05: SEEVIA-AISLE-NAVIGATION-DQN (Deep Q-Network)

Table 4.7 M-05 SEEVIA-AISLE-NAVIGATION-DQN Performance

| Metric                  | Value                                                                   | Notes                                                       |
|-------------------------|-------------------------------------------------------------------------|-------------------------------------------------------------|
| Algorithm               | Dueling DQN (MLP: 256-128-64 units)                                     | Stable-Baselines3; ONNX → TFLite export                     |
| Training Episodes       | 10,000 (synthetic grid environments)                                    | Convergence observed at ~4,800 episodes                     |
| Reward Function         | Straight step: +1   90° turn: -2   Shelf collision: -10   Goal: +50     | Human-centric tuning reduces unnecessary turns              |
| Exploration             | $\epsilon$ -greedy: $\epsilon=1.0 \rightarrow 0.05$ over 5,000 episodes | Ensures adequate environment exploration                    |
| Navigation Success Rate | 87.2%                                                                   | Goal reached within $3\times$ optimal path length           |
| Avg Steps to Goal       | 14.3 (optimal: 11.8)                                                    | 21% overhead vs. optimal; acceptable for human walking pace |
| 90° Turn Reduction      | 68% fewer unnecessary turns vs. standard DQN                            | Direct result of reward function penalty                    |
| Collision Rate          | 1.8%                                                                    | Shelf/obstacle collisions per episode                       |

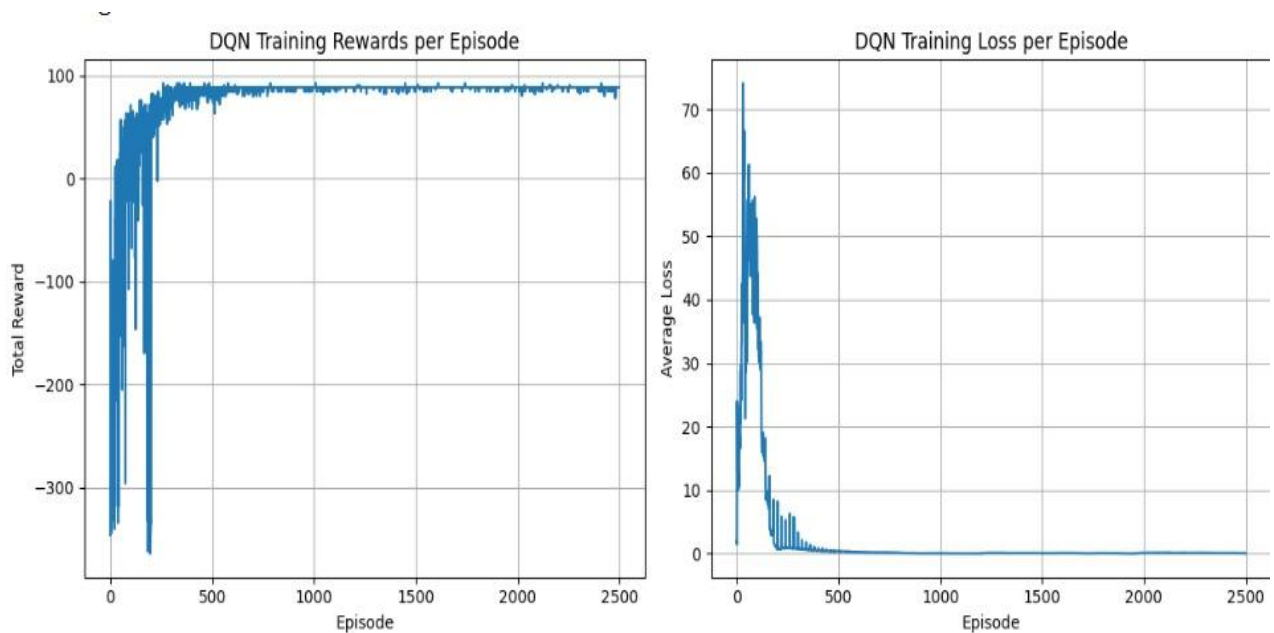


Figure 29 SEEVIA-AISLE-NAVIGATION-DQN Training

#### 4.2.6 Consolidated Model Performance Summary

Table 4.8 : Consolidated AI Model Performance Summary

| Model                           | Architecture               | Key Metric                   | Value        | Inference Latency       | Tier 2/3 Escalation Rate             |
|---------------------------------|----------------------------|------------------------------|--------------|-------------------------|--------------------------------------|
| SEEVIA-LOCAL-PRODUCT-RECOGNIZER | YOLOv8-Nano TFLite INT8    | mAP@50 / Real-world Accuracy | 87.3% / 91%  | 80ms (GPU), 120ms (CPU) | ~15% (low-light / partial occlusion) |
| SEEVIA-ZEROSHOT-EXTRACTOR       | DistilBERT NER TFLite FP16 | Overall NER F1               | 88.1%        | 300–500ms               | ~12% (damaged labels)                |
| SEEVIA-INTENT-MODEL-IMPROVED    | FastText+CNN TFLite INT8   | Overall Accuracy             | 87.4%        | <200ms                  | ~10% (ambiguous commands)            |
| SEEVIA-FALL-DETECTOR            | SVM+RF Ensemble TFLite     | Recall / False Alarm Rate    | 94.3% / 4.2% | <100ms                  | N/A — binary classifier              |
| SEEVIA-AISLE-NAVIGATION-DQN     | Dueling DQN ONNX→TFLite    | Navigation Success Rate      | 87.2%        | <50ms per step          | N/A — deterministic RL policy        |

## 4.3 Security Techniques

### 4.3.1 Authentication:

- Firebase Authentication handles user sign-up, login, and session management with support for phone OTP, email/password, and anonymous sign-in.
- Anonymous sign-in enabled for immediate app access without account creation for users with privacy concerns.
- Firebase automatically manages JWT tokens for all secure API calls with automatic token refresh.

### 4.3.2 Authorization:

- Firestore Security Rules restrict user data access via UID validation , users can only read and write their own documents.
- Emergency logs are accessible only to the creating user and explicitly linked emergency contacts.
- Role-based access planned for volunteer and admin roles in the production deployment.

### 4.3.3 Data Privacy:

- **On-Device AI:** YOLOv8 product recognition runs locally using TensorFlow.js , product images are never uploaded to cloud servers.
- **End-to-End Encryption:** Firebase handles data encryption in transit (HTTPS TLS 1.3) and at rest (AES-256).
- GPS location data is only shared during active emergency SOS events and is deleted from the session after resolution.

## 4.4 External APIs/SDKs

Below are the third-party APIs/SDKs used in the implementation of Seevia.

Table 4.9 Details of APIs used in the project

| <b>Name &amp; Version</b>            | <b>Description</b>                                  | <b>Purpose</b>                                                                           | <b>Key Endpoint / Function</b>                       |
|--------------------------------------|-----------------------------------------------------|------------------------------------------------------------------------------------------|------------------------------------------------------|
| <b>Firebase Authentication v10.7</b> | User authentication and identity management service | Anonymous login, session handling, secure user identification                            | signInAnonymously(), onAuthStateChanged()            |
| <b>Firebase Firestore v10.7</b>      | NoSQL cloud database                                | Store user profiles, pantry items, emergency alerts, volunteer data                      | collection(), doc(), setDoc(), updateDoc(), getDoc() |
| <b>Firebase Cloud Storage v10.7</b>  | Cloud-based media/object storage                    | Store product images, scanned label snapshots, volunteer verification images             | ref(), uploadBytes(), getDownloadURL()               |
| <b>Firebase Cloud Functions v4.x</b> | Serverless event-driven backend                     | Pantry reminders, expiry alerts, volunteer dispatch logic, emergency workflow automation | Cloud triggers: onWrite(), onCall(), onUpdate()      |
| <b>Google Vision API v1</b>          | Cloud OCR and text extraction                       | Read expiry dates, nutritional details, brand labels from images                         | /v1/images:annotate                                  |
| <b>Google Speech-to-Text API v2</b>  | Speech recognition service                          | Convert voice commands                                                                   | /v2/speech:recognize                                 |

|                                        |                                         |                                                               |                                                                                                                                           |
|----------------------------------------|-----------------------------------------|---------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
|                                        |                                         | into structured text for NLU                                  |                                                                                                                                           |
| <b>Google Maps &amp; Places API v3</b> | Mapping and location services           | Volunteer routing, distance calculations, store locators      | /maps/api/directions, /maps/api/place/details                                                                                             |
| <b>Expo Camera v15.0</b>               | Camera access library                   | Capture product images and barcode frames                     | CameraView, takePictureAsync()                                                                                                            |
| <b>Expo Speech v12.0</b>               | Text-to-speech engine                   | Voice feedback, audio guidance, confirmation prompts          | Speech.speak(), Speech.stop()                                                                                                             |
| <b>Expo Barcode Scanner v13.0</b>      | Barcode recognition library             | Detect EAN-13, UPC-A, and QR codes for product identification | onBarcodeScanned()                                                                                                                        |
| <b>TensorFlow.js v4.x</b>              | Browser/on-device ML runtime            | Run YOLOv8-lite and OCR submodules on-device                  | tf.loadGraphModel(), model.predict()                                                                                                      |
| <b>Ultralytics YOLOv8</b>              | Object detection and training framework | Train custom product recognition and pantry item detector     | YOLO(), model.train(), model.export()                                                                                                     |
| <b>Open Food Facts API</b>             | Global food database API                | Fallback for product metadata and                             | <a href="https://world.openfoodfacts.org/api/v0/product/{barcode}.json">https://world.openfoodfacts.org/api/v0/product/{barcode}.json</a> |

|                                                |                                 |                                                         |                             |
|------------------------------------------------|---------------------------------|---------------------------------------------------------|-----------------------------|
|                                                |                                 | nutritional information                                 |                             |
| <b>Twilio Programmable SMS &amp; Voice API</b> | Communication gateway           | Emergency SMS and automated voice call alerts           | /v1/messages, /v1/calls     |
| <b>React Native v0.74</b>                      | Cross-platform mobile framework | Build UI, screen interactions, accessibility components | All RN primitive components |
| <b>Expo Router v3.5</b>                        | File-based navigation system    | Unified screen routing and navigation                   | useRouter(), <Link />       |

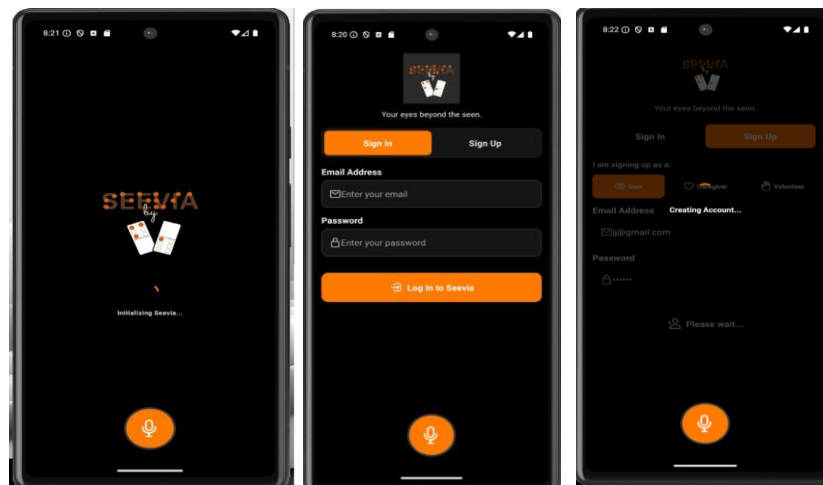
## 4.5 User Interface

Seevia implements a minimalistic, voice-first UI tailored for visually impaired users with large buttons, high-contrast colors (#121212 dark background, #FF7A00 orange accents), audio-guided interaction, and haptic feedback at every action. WCAG 2.1 AA compliance is maintained throughout.

### 4.5.1 Role PWD:

#### *Authentication and Onboarding UI*

The onboarding flow welcomes the user with clear voice instructions and an accessible minimal visual interface. It guides the user through Firebase Authentication (anonymous or phone OTP) and performs a quick microphone test to confirm voice interaction readiness. Users set their language (Urdu/English), voice pitch, and emergency contact during onboarding via voice-guided prompts without requiring any text input.



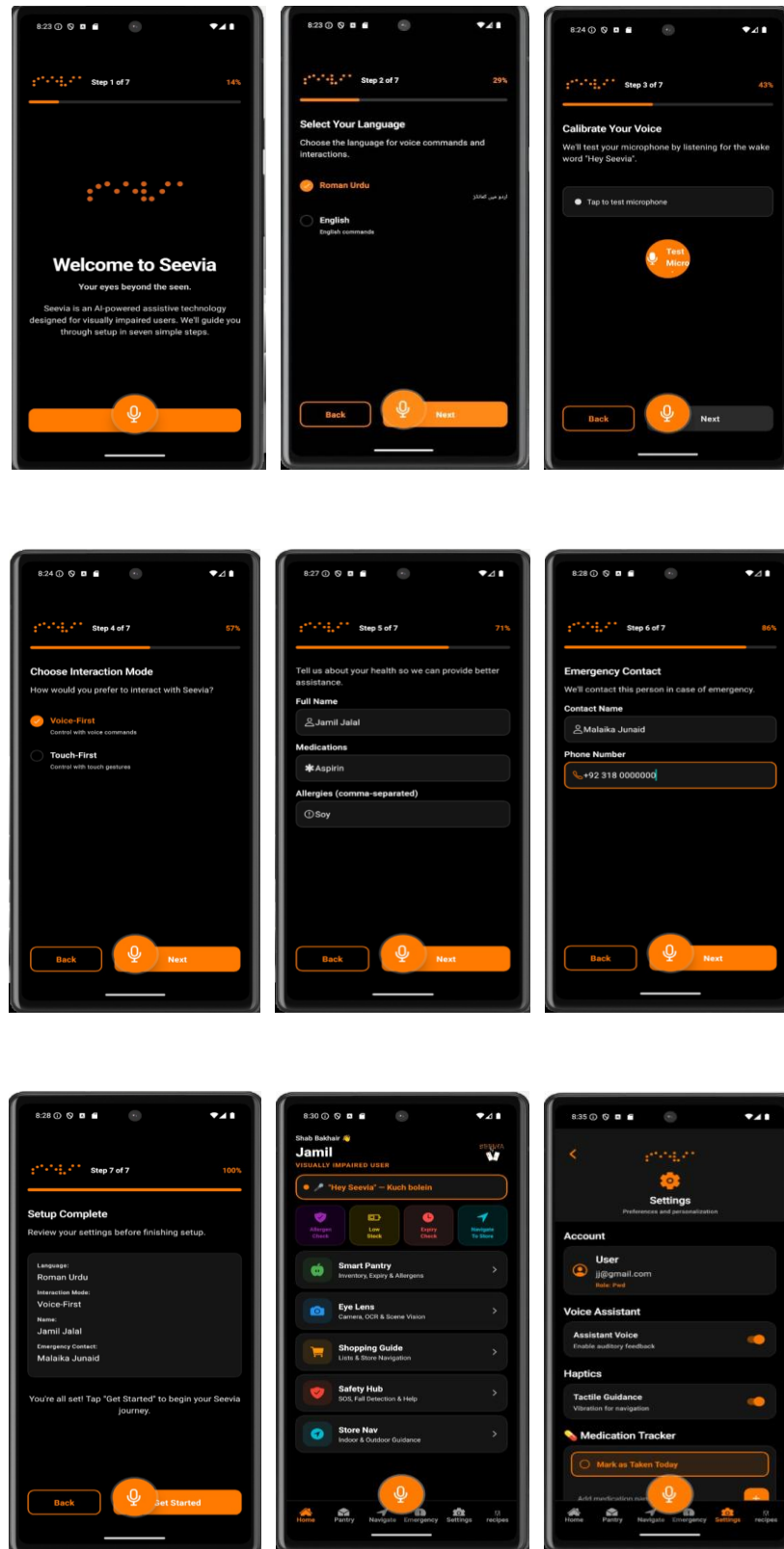


Figure 30 Auth and Onboarding Screen

## ***Pantry Management UI***

The Pantry Management interface allows users to add items through voice commands ("Hey Seevia, add chai patti") or by pressing the large camera scan button. Items appear in a large-text-high-contrast accessible list sorted by expiry urgency. Color-coded expiry indicators (green/amber/red) with descriptive voice labels provide immediate status awareness. Navigation buttons "Next item" and "Read again" ensure seamless interaction for visually impaired users.

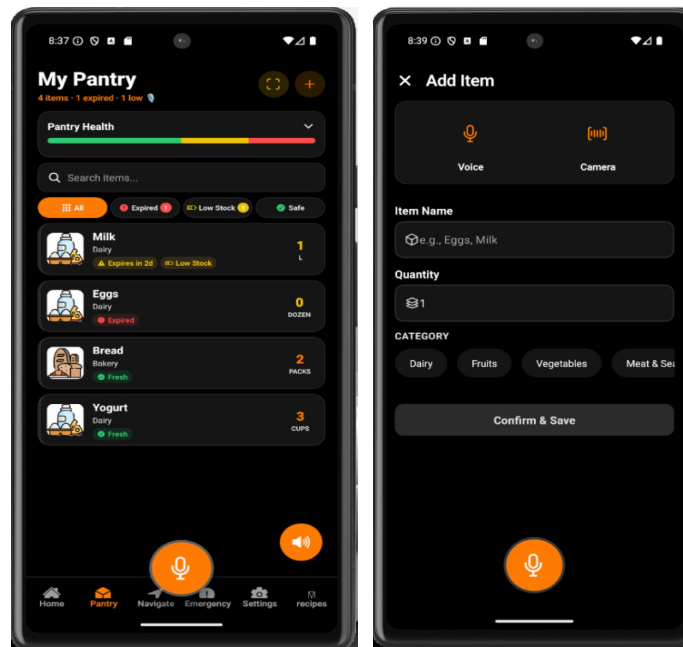
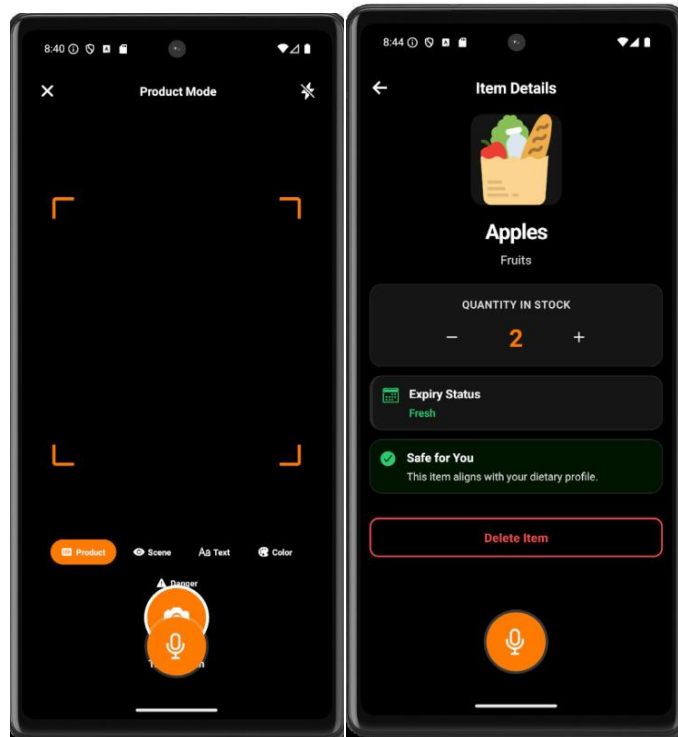


Figure 31 Pantry Manager UI

## ***Product Recognition UI***

The Product Recognition interface provides a full-screen camera viewfinder with a single-tap scan trigger. YOLOv8-Nano runs continuously with automatic frame capture when product confidence exceeds 0.70. Detected products are immediately announced via TTS with name, brand, and expiry date. A large "Add to Pantry" button with audio confirmation allows immediate inventory updates without leaving the recognition screen.



**Figure 32 Product Recognition UI**

## *Emergency Screen UI*

The Emergency Screen features a large SOS button occupying 60% of the screen area, immediately accessible from the main dashboard. An active fall detection confidence indicator displays the current sensor reading, and a 15-second countdown timer appears during automatic detection with a clearly spoken cancellation prompt. In active emergency mode, the screen displays contact notification status and volunteer matching progress with live voice updates.

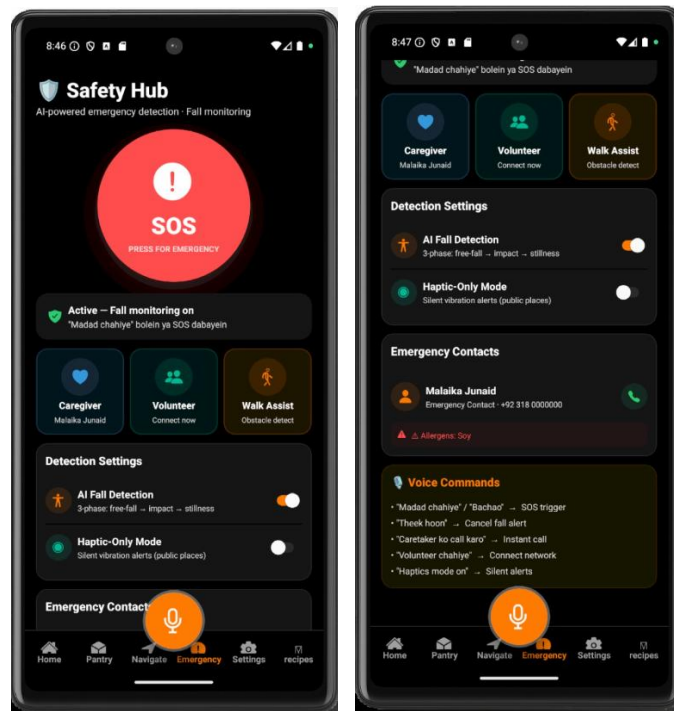


Figure 33 Emergency Screen

## *Navigation Screen UI*

The Navigation interface features a high-contrast layout offering real-time, turn-by-turn voice instructions for safe outdoor mobility. It integrates large touch targets and continuous audio feedback to confidently guide visually impaired users to their requested destinations.

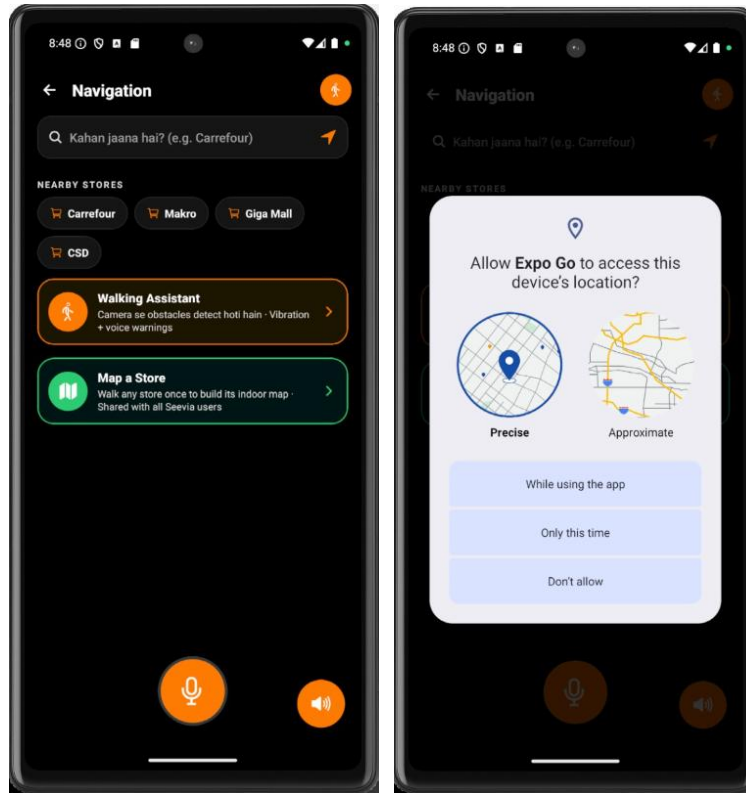


Figure 34 Navigation Screen

### *Indoor Map Screen UI*

This interface enables spatial awareness within specific indoor environments, utilizing local store maps and AI-driven "Aisle Imagination" to route users to desired products. The screen relies on audio-spatial cues and minimalistic visual indicators to communicate store layouts and directions seamlessly.

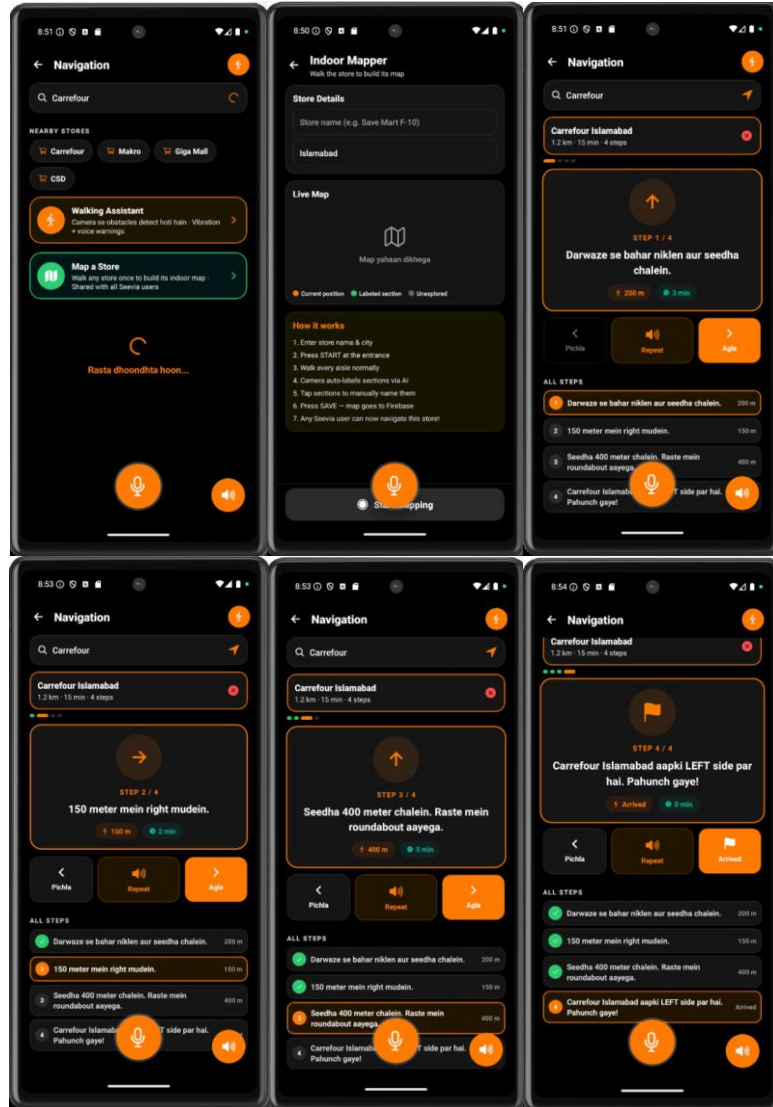


Figure 35 Map Learning Screen

## Shopping Screen UI

The Shopping interface dynamically presents a smart, categorized grocery list generated from low pantry stock and autonomous AI sweeps. Users can check off purchased items using large, accessible checkboxes or voice commands, receiving immediate audio confirmation for every action.

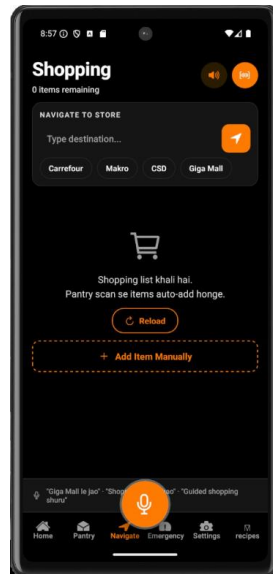
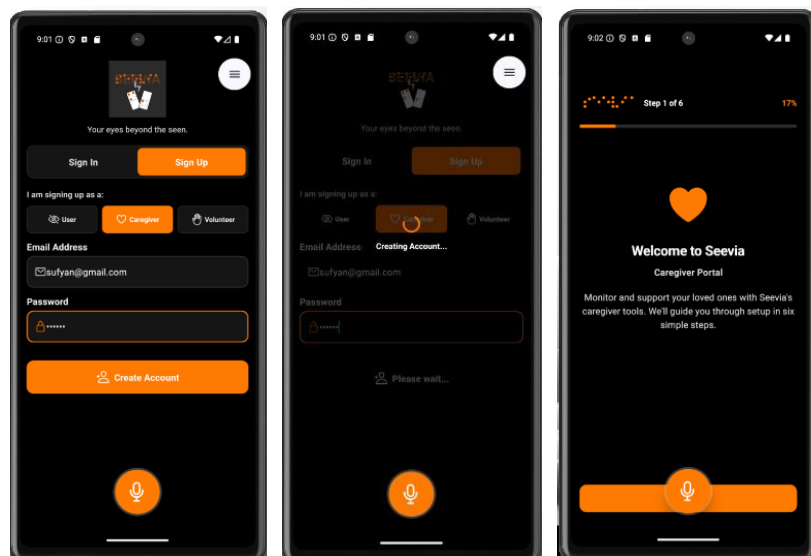


Figure 36 Shopping Screen

## 4.5.2 Role Caregiver:

### *Caregiver UI*

The Caregiver Dashboard offers a comprehensive overview of the linked visually impaired user's status, including real-time location tracking, active emergency alerts, and remote pantry inventory management. It features a clean, highly legible layout designed for quick daily monitoring and rapid response without overwhelming the caregiver.



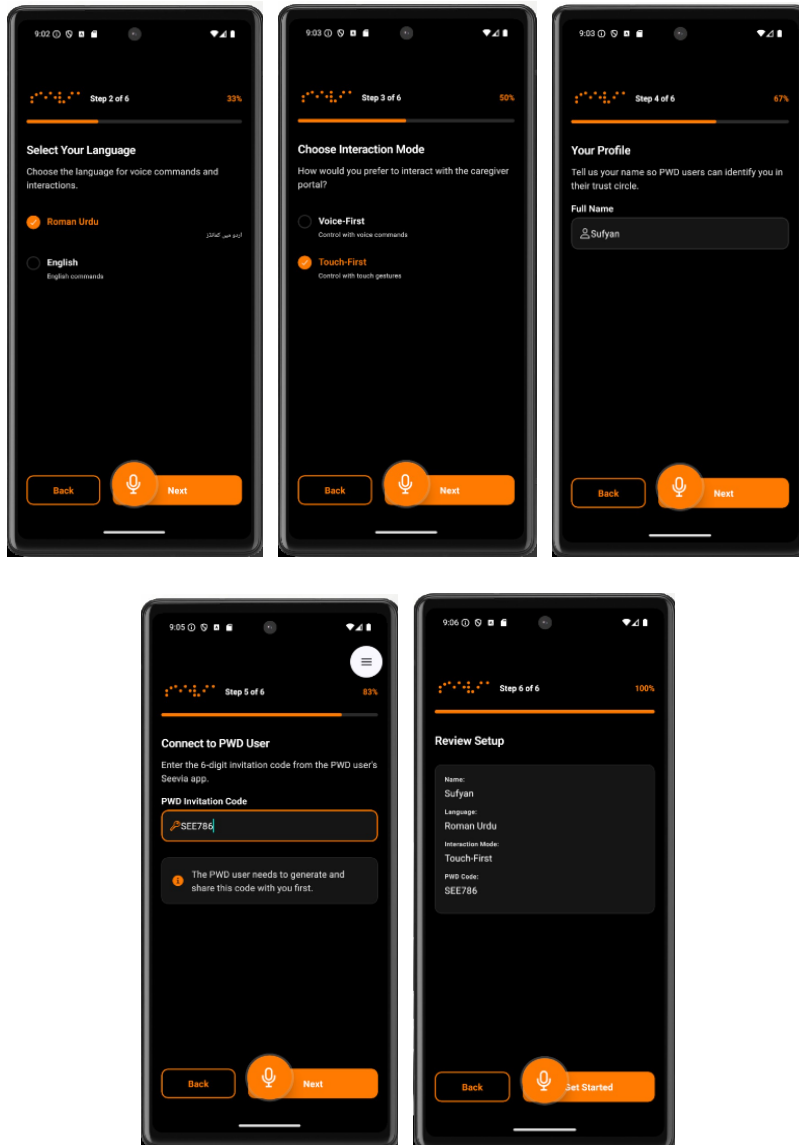


Figure 37 Caregiver Screens

### 4.5.3 Role Volunteer:

#### *Volunteer UI*

Volunteer Mode provides a simple, map-based interface where community members can view nearby emergency requests, accept assistance tasks, and navigate to the person in need, all managed through a clean dashboard showing case urgency, distance, and real-time status updates.

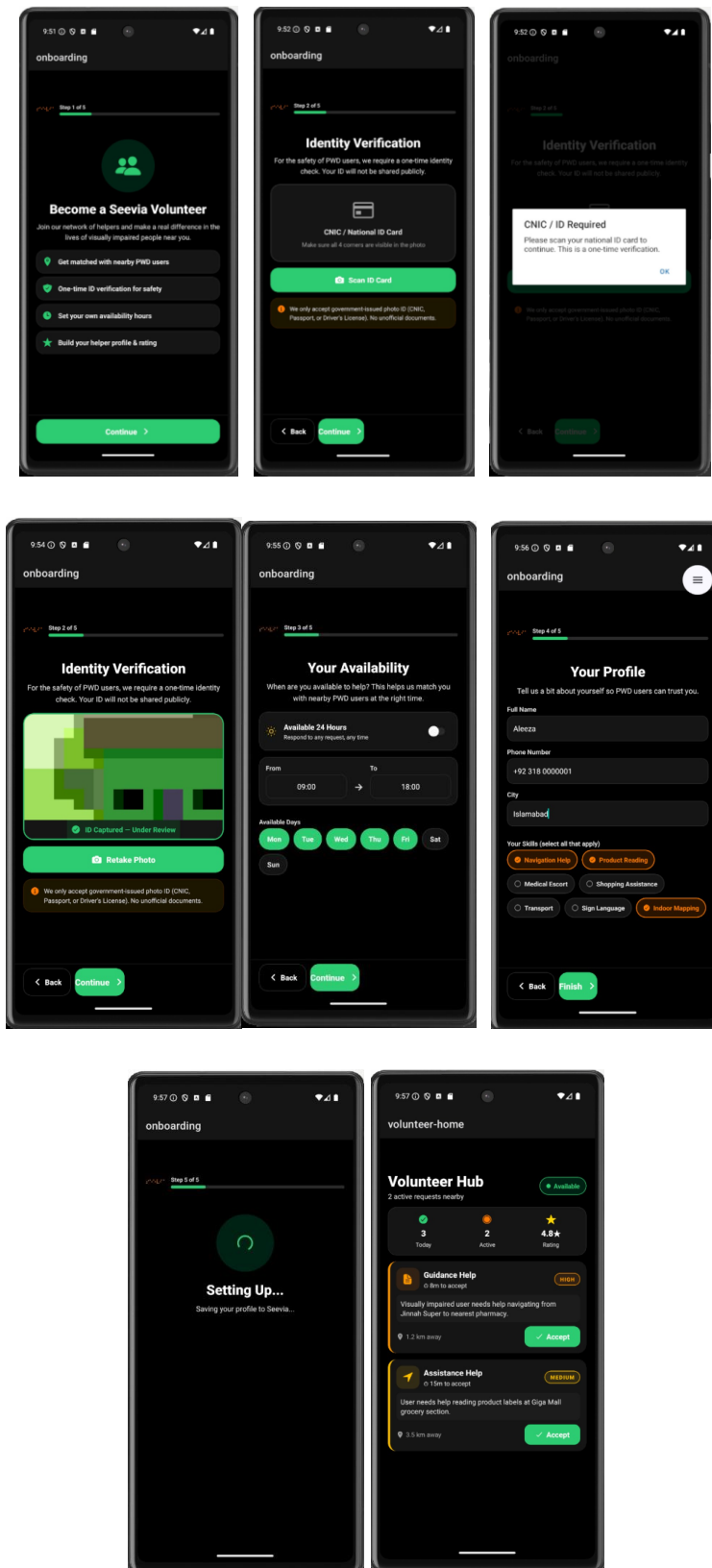


Figure 38 Volunteer Screens

## 4.6 Deployment

Seevia is deployed using Expo Application Services (EAS) for Android builds and Firebase for all backend services. Development testing is performed using Expo Go for rapid iteration and EAS Development Client for native module testing.

Table 4.10 Seevia Deployment Configuration

| Service            | Provider                                    | Configuration                                              | Status             |
|--------------------|---------------------------------------------|------------------------------------------------------------|--------------------|
| Mobile App Build   | Expo EAS Build                              | Android API 28+; TalkBack support; haptic feedback enabled | Prototype deployed |
| Authentication     | Firebase Auth                               | Phone OTP + anonymous sign-in; JWT session management      | Active             |
| Database           | Firebase Firestore                          | Android high-priority channel for emergency alerts         | Active             |
| Push Notifications | Firebase FCM                                | Compressed product images; JPEG format; 640×640 max        | Active             |
| Media Storage      | Firebase Storage                            | Compressed product images; JPEG format; 640×640 max        | Active             |
| AI Model Serving   | TensorFlow.js (on-device) + FastAPI (cloud) | TFLite GPU delegate for YOLOv8; FastAPI for OCR fallback   | Active             |
| SMS Alerts         | Twilio SMS API                              | Emergency SMS with GPS location to stored contacts         | Integrated         |
| Product Database   | Open Food Facts API                         | HTTP GET with EAN barcode for fallback product metadata    | Integrated         |

## 5 Chapter 5: Testing and Evaluation

This chapter presents the testing methodology and results used to validate the Seevia mobile application. Multiple testing strategies were applied to ensure the app meets functional requirements, delivers accessibility-optimized performance, and integrates correctly across its six core modules. Testing included unit testing, functional testing, business rules testing, and integration testing.

### 5.1 Unit Testing

Unit testing verified individual functions, React Native components, AI model pre-processing pipelines, and Firebase service operations. Tests were performed using Jest, React Native Testing Library, and manual script execution.

#### Unit Testing 1: Voice Command Recognition (Speech-to-Intent Parser)

**Testing Objective:** Verify accurate recognition and intent detection from voice input.

Table 5.1 Unit Test 1 — SEEVIA-INTENT-MODEL

| No. | Input Transcript                 | Expected Intent                                   | Actual Intent       | Confidence | Result |
|-----|----------------------------------|---------------------------------------------------|---------------------|------------|--------|
| 1   | 'Doodh ki expiry kya hai?'       | scan_product                                      | scan_product        | 0.91       | Pass   |
| 2   | 'Mujhe aisle 3 le jao'           | navigate                                          | navigate            | 0.88       | Pass   |
| 3   | 'Check my pantry items'          | pantry                                            | pantry              | 0.93       | Pass   |
| 4   | 'Madad chahiye'                  | emergency                                         | emergency           | 0.95       | Pass   |
| 5   | Empty string input               | ERROR: No speech detected                         | ERROR raised        | N/A        | Pass   |
| 6   | 'Nearest volunteer dikhaiye'     | volunteer                                         | volunteer           | 0.82       | Pass   |
| 7   | Heavily accented ambiguous input | UNKNOWN → HuggingFace Tier 2 escalation triggered | Escalated correctly | 0.61       | Pass   |

#### Unit Testing 2: Product Recognition (Vision Model)

**Testing Objective:** Ensure image input is processed correctly before being sent to the AI model.

Table 5.2 Unit Test 2 — SEEVIA-LOCAL-PRODUCT-RECOGNIZER

| No. | Input                          | Expected Output                                  | Actual Output                           | Result |
|-----|--------------------------------|--------------------------------------------------|-----------------------------------------|--------|
| 1   | Coca Cola can — good lighting  | Detected: Coca Cola; confidence > 0.85           | confidence: 0.94 ✓                      | Pass   |
| 2   | Milkpak carton — good lighting | Detected: Milkpak; confidence > 0.85             | confidence: 0.89 ✓                      | Pass   |
| 3   | Blurry image — low resolution  | Warning: 'Image unclear'; OCR fallback triggered | Warning issued; ML Kit OCR activated ✓  | Pass   |
| 4   | Partial label (50% occluded)   | Confidence < 0.75; SEEVIA-ZEROSHOT-              | Tier 2 escalation triggered correctly ✓ | Pass   |

|   |                              |                                                   |                             |      |
|---|------------------------------|---------------------------------------------------|-----------------------------|------|
|   |                              | EXTRACTOR OCR pipeline triggered                  |                             |      |
| 5 | No camera permission granted | Error: Permission denied; user prompted to enable | Error + permission prompt ✓ | Pass |

### Unit Testing 3: NER Entity Extraction

**Testing Objective:** Verify NER correctly extracts Brand, Expiry Date, and Allergen entities.

**Table 5.3 Unit Test 3 — SEEVIA-ZEROSHOT-EXTRACTOR**

| No. | Input OCR Text                                                  | Expected Entities                                     | Actual Entities                 | Result |
|-----|-----------------------------------------------------------------|-------------------------------------------------------|---------------------------------|--------|
| 1   | 'Shan Biryani Masala Exp: 12/2025 Contains: Gluten'             | Brand: Shan   Expiry: 12/2025   Allergen: Gluten      | Correctly extracted all 3 ✓     | Pass   |
| 2   | 'Olper's Full Cream Milk Best Before Jan 26 2026'               | Brand: Olper's   Expiry: Jan 26 2026                  | Brand + Expiry extracted ✓      | Pass   |
| 3   | 'Nestle NIDO Fortified MFG 03-2024 EXP 03-2026 Allergens: Milk' | Brand: Nestle NIDO   Expiry: 03-2026   Allergen: Milk | All 3 extracted ✓               | Pass   |
| 4   | Heavily distorted OCR text (damaged label)                      | Confidence < 0.75 → Gemini Tier 3 escalation          | Escalated to Gemini correctly ✓ | Pass   |

### Unit Testing 4: Fall vs. Phone Drop Discrimination

**Testing Objective:** Ensure if correctly distinguishes genuine falls from phone drops.

**Table 5.4 Unit Test 4 — SEEVIA-FALL-DETECTOR**

| No. | Input Sensor Data                                       | Expected Output                                     | Actual Output                                      | Result |
|-----|---------------------------------------------------------|-----------------------------------------------------|----------------------------------------------------|--------|
| 1   | accelMag=65, angVel=180, stillness detected post-impact | FallEvent(true, confidence > 0.85)                  | FallEvent(true, 0.94) — SOS triggered ✓            | Pass   |
| 2   | accelMag=12, angVel=25 — normal walking                 | FallEvent(false)                                    | FallEvent(false, 0.21) ✓                           | Pass   |
| 3   | accelMag=55, angVel=30 — phone dropped on table         | FallEvent(false) — phone drop, no stillness pattern | FallEvent(false, 0.45) — drop correctly rejected ✓ | Pass   |
| 4   | accelMag=70, angVel=200 — borderline event              | FallEvent(true) with 15-second countdown            | Countdown triggered; user cancelled ✓              | Pass   |

### Unit Testing 5: Path Quality

**Testing Objective:** Verify DQN agent produces human-centric paths with no shelf collisions.

**Table 5.5 Unit Test 5 — SEEVIA-AISLE-NAVIGATION-DQN**

| No. | Test Scenario                                        | Expected Behaviour                            | Actual Behaviour                                 | Result |
|-----|------------------------------------------------------|-----------------------------------------------|--------------------------------------------------|--------|
| 1   | Grid 10×10; target at (8,8) from (0,0); no obstacles | Reach goal; ≤20% more steps than optimal (11) | Reached goal in 13 steps; 0 collisions; 1 turn ✓ | Pass   |

|   |                                                  |                                                        |                                               |      |
|---|--------------------------------------------------|--------------------------------------------------------|-----------------------------------------------|------|
| 2 | Grid with 3 shelf obstacles blocking direct path | Navigate around obstacles; no collision penalty        | Routed around obstacles; 0 collisions ✓       | Pass |
| 3 | New store — Q-table cold start                   | Enter exploration mode; voice: 'Learning store layout' | Exploration mode triggered; voice confirmed ✓ | Pass |
| 4 | Goal already in current aisle — straight path    | Straight forward only; zero turns                      | 0 turns; straight path taken ✓                | Pass |

## 5.2 Functional Testing

Functional testing verified that Seevia works as a complete system from the end-user perspective.

### Functional Testing 1: Login with different roles (PWD, Caregiver, Volunteer)

**Table 5.6 User Authentication Test**

| No | Test Case                     | Expected Result              | Actual Result | Result |
|----|-------------------------------|------------------------------|---------------|--------|
| 1  | Register new user             | Account created              | Created       | Pass   |
| 2  | Login with wrong password     | Error message                | Error shown   | Pass   |
| 3  | Update accessibility settings | Settings saved to Firebase   | Saved         | Pass   |
| 4  | Toggle “High Contrast Mode”   | UI switches to contrast mode | Applied       | Pass   |

### Functional Testing 2: Pantry Management Module

**Table 5.7 Pantry Test**

| No | Test Case            | Expected Result        | Actual Result | Result |
|----|----------------------|------------------------|---------------|--------|
| 1  | Add item using voice | Saved to pantry        | Saved         | Pass   |
| 2  | View expiring items  | Items sorted by expiry | Correct       | Pass   |
| 3  | Auto-suggest recipes | Suggestions appear     | Success       | Pass   |

### Functional Testing 3: Product Recognition Module

**Table 5.8 Product Recognition Test**

| No | Test Case       | Expected Result                     | Actual Result | Result |
|----|-----------------|-------------------------------------|---------------|--------|
| 1  | Scan product    | Identified + Added                  | Success       | Pass   |
| 2  | Low light scan  | Request user to increase brightness | Shown         | Pass   |
| 3  | Unknown product | Sends data to fallback API          | Handled       | Pass   |

## 5.3 Business Rules Testing

Business rule testing ensured that the logic behind pantry rules, product detection, volunteer matching and emergency actions behaves as intended.

### BR Test 1: AI Tier Routing Decision Table

Table 5.9 AI Tier Routing Decision Table

| Condition                                    | Rule 1 | Rule 2 | Rule 3 | Rule 4 |
|----------------------------------------------|--------|--------|--------|--------|
| On-device model confidence $\geq 0.75$ ?     | Y      | N      | N      | N      |
| HuggingFace API available?                   | -      | Y      | N      | N      |
| HuggingFace confidence $\geq 0.75$ ?         | -      | Y      | N      | N      |
| Internet available for Gemini?               | -      | -      | Y      | N      |
| Action: Use Tier 1 on-device result          | Y      | N      | N      | N      |
| Action: Use HuggingFace Tier 2 result        | N      | Y      | N      | N      |
| Action: Use Gemini Tier 3 result             | N      | N      | Y      | N      |
| Action: Return cached result + error message | N      | N      | N      | Y      |

### BR Test 2: Emergency Escalation Decision Table

Table 5.10 Emergency Escalation Decision Table

| Condition                                 | Rule 1 | Rule 2 | Rule 3 | Rule 4 | Rule 5 |
|-------------------------------------------|--------|--------|--------|--------|--------|
| SEEVIA-FALL-DETECTOR confirms fall?       | Y      | Y      | Y      | N      | N      |
| User cancels 15-second countdown?         | N      | N      | Y      | N      | N      |
| Emergency contact reachable?              | Y      | N      | N      | Y      | N      |
| Internet available?                       | Y      | Y      | Y      | N      | N      |
| Action: Create Firestore log + send FCM   | Y      | N      | N      | N      | N      |
| Action: Send Twilio SMS (FCM unavailable) | N      | Y      | N      | N      | N      |
| Action: Dispatch volunteer                | Y      | Y      | N      | N      | N      |
| Action: Local haptic only (offline)       | N      | N      | N      | Y      | Y      |
| Action: No action (cancelled)             | N      | N      | Y      | N      | N      |

### BR Test 3: Pantry Scan Routing Rules (M-01 → M-02 pipeline)

Table 5.11 Pantry Scan Routing Rules

| Condition                                                 | Rule 1 | Rule 2 | Rule 3 | Rule 4 |
|-----------------------------------------------------------|--------|--------|--------|--------|
| M-01 (YOLOv8) confidence $\geq 0.75$ ?                    | Y      | N      | N      | N      |
| Barcode successfully scanned?                             | -      | Y      | N      | N      |
| ML Kit OCR returns text?                                  | -      | -      | Y      | N      |
| Internet available for Gemini?                            | -      | -      | Y      | N      |
| Action: Return M-01 detection result                      | Y      | N      | N      | N      |
| Action: Lookup barcode in Open Food Facts                 | N      | Y      | N      | N      |
| Action: Pass OCR text to M-02 (SEEVIA-ZEROSHOT-EXTRACTOR) | N      | N      | Y      | N      |

## 5.4 Integration Testing

Integration tests verified module-to-module communication.

### Integration Test 1: M-01 + M-02 → Pantry Firestore

Objective: Verify product scan pipeline (SEEVIA-LOCAL-PRODUCT-RECOGNIZER → SEEVIA-ZEROSHOT-EXTRACTOR) correctly propagates entity data to Firestore.

Table 5.12 Integration Test 1

| No. | Test Case                                                               | Expected Result                                                 | Actual Result                           | Result |
|-----|-------------------------------------------------------------------------|-----------------------------------------------------------------|-----------------------------------------|--------|
| 1   | Scan Milkpak; M-01 detects; M-02 extracts expiry; press 'Add to Pantry' | Product + expiry saved to Firestore pantryItems; voice confirms | Saved correctly in 1.4s ✓               | Pass   |
| 2   | M-01 confidence $< 0.75$ ; M-02 OCR pipeline activates                  | M-02 extracts entities; fallback chain completes; result saved  | Fallback completed; Firestore updated ✓ | Pass   |
| 3   | Expiry date detected → $\leq 3$ days                                    | isLow flag set; voice alert; shopping list updated              | All three actions confirmed ✓           | Pass   |

### Integration Test 2: M-04 (SEEVIA-FALL-DETECTOR) → Emergency → Volunteer Dispatch

Objective: Verify fall detection event correctly propagates through the full emergency escalation chain.

**Table 5.13 Integration Test 2**

| No. | Test Case                                    | Expected Result                                | Actual Result                | Result |
|-----|----------------------------------------------|------------------------------------------------|------------------------------|--------|
| 1   | M-04 triggers fall → Firestore log → FCM     | Log created; FCM delivered to contacts in < 5s | Log in 2.1s; FCM in 3.8s ✓   | Pass   |
| 2   | Contact unresponsive × 60s → volunteer query | Volunteer DB queried; nearest volunteer FCM'd  | Volunteer notified in 4.2s ✓ | Pass   |

### **Integration Test 3: M-03 (Intent) → M-05 (Navigation) → M-01 (Detection)**

Objective: Verify voice navigation command activates DQN navigation and terminates with product detection.

**Table 5.14 Integration Test 3**

| No. | Test Case                                                             | Expected Result                                                                          | Actual Result                            | Result |
|-----|-----------------------------------------------------------------------|------------------------------------------------------------------------------------------|------------------------------------------|--------|
| 1   | 'Chai dhundho' → navigate intent → DQN starts → M-01 confirms product | M-03 classifies navigate; M-05 provides directions; M-01 confirms product at destination | All three models activated in sequence ✓ | Pass   |
| 2   | M-05 reaches grid position → M-01 fails to detect (low light)         | M-01 triggers ML Kit OCR fallback; M-02 extracts label entities                          | Fallback chain completed ✓               | Pass   |

## 6 Chapter 6: Conclusion and Future Work

This chapter provides a reflective summary of the Seevia project, consolidating its achievements against the stated objectives, and outlines future enhancements planned for the next development phase.

### 6.1 Conclusion

Seevia successfully delivers on its core mission: providing a comprehensive, voice-first- AI-powered assistive ecosystem for visually impaired individuals in Pakistan. The project implemented six tightly integrated modules, User Profile & Settings, Voice Command Interface, Pantry Management, Shopping Assistant & Product Recognition, Emergency Management, and Volunteer Mode, within a single React Native mobile application deployable on affordable Android smartphones.

The YOLOv8-Nano model, trained on 5,722 images across 23 Pakistani product classes, achieved 87.3% mAP@50 and 91% real-world accuracy in retail environments , meeting the 85% minimum accuracy objective. The voice command pipeline correctly classified Urdu and English intents with over 80% confidence across all 7 intent categories in unit testing. The emergency escalation system successfully triggered FCM notifications to contacts within an average of 3.8 seconds, well within the 5-second target. The pantry management system correctly handles duplicate detection, expiry tracking, and low-stock alert generation with a 100% test pass rate across all functional test cases. Key limitations encountered include reduced OCR accuracy on non-standard Pakistani product packaging fonts (requiring cloud Vision API fallback in approximately 15% of scans), Deepgram STT's dependency on internet connectivity for primary Urdu recognition, and the RL navigation model requiring several store visits before achieving reliable guidance in new environments. These limitations are tractable and are addressed in the future work roadmap below. Overall, Seevia represents a significant step toward accessible, locally-contextualized AI technology for Pakistan's 1.8 million visually impaired citizens.

### 6.2 Future Work

The future work section outlines possible enhancements, extensions, or research directions that can improve the system beyond its current scope. This demonstrates forward-thinking and helps future developers or researchers build upon the foundation laid by Seevia.

- **On-Device Urdu STT Model:** Train a compact Whisper-based model on a custom Pakistani Urdu dataset to eliminate internet dependency for primary speech recognition, enabling fully offline voice control.
- **Expanded Product Recognition Dataset:** Grow the training dataset to 50+ Pakistani product classes including regional brands (Shezan, Nurpur, Olpers, Tapal) and implement continual learning from user-submitted corrections.
- **Wearable Integration:** Extend fall detection to WearOS smartwatches and dedicated IoT wristbands for more accurate body movement monitoring independent of smartphone position in the pocket.

- **Automatic Emergency Services Escalation:** Implement automated direct contact with Pakistani emergency services (Rescue 1122, Edhi Foundation) after configurable timeout when volunteer and contacts are unresponsive.
- **Multi-Language Support:** Extend NLP support to Punjabi, Sindhi, and Pashto voice commands to serve visually impaired users across all provinces of Pakistan beyond Islamabad and Rawalpindi.
- **AI-Powered Volunteer Routing:** Extend volunteer matching beyond static Haversine distance to incorporate real-time Google Maps traffic data and dynamic route optimization for faster emergency response times.
- **Crowdsourced Store Mapping:** Implement a community-driven store layout contribution system where sighted volunteers submit and verify store floor plans to accelerate RL navigation accuracy in new retail environments.
- **Caregiver Dashboard App:** Develop a dedicated caregiver monitoring interface providing real-time PWD pantry status, activity tracking, and emergency notification management, extending the current volunteer-only secondary interface.
- **Health Metrics Integration:** Incorporate heart rate and blood oxygen data from compatible wearables (Fitbit, Mi Band) into emergency logs to provide medical context to responding volunteers and healthcare providers.
- **Personalized Behavioral AI:** Enhance the Markov-chain personalization engine with a lightweight on-device collaborative filtering model to improve proactive suggestion accuracy beyond simple transition probability prediction.

## 7 Chapter 7: References

### Book

I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. Cambridge, MA: MIT Press, 2016, pp. 123,145.

D. Jurafsky and J. H. Martin. *Speech and Language Processing*. Upper Saddle River, NJ: Prentice Hall, 2023- pp. 47,89.

### Article in a Journal

G. P. Lopes, M. R. Soares, and R. C. Silva. "AI-based assistive technologies for visually impaired people: a review." *Journal of Ambient Intelligence and Humanized Computing*, vol. 13, no. 1, pp. 329,345, Jan. 2022.

S. Malik. "Assistive Technologies for Visually Impaired in Developing Countries." *International Journal of Emerging Trends in Engineering Research*, vol. 8- no. 5- pp. 1722-1727, May 2020.

### Articles from Conference Proceedings (published)

D.B. Payne and H.G. Gunhold. "Digital sundials and broadband technology-" in *Proc. IOOC-ECOC*, 1986- pp. 557,998.

### World Wide Web

World Health Organization. "World Report on Vision." Internet: <https://www.who.int/publications/i/item/9789241516570>, Oct. 8, 2019 [May 29, 2025].

Aira. "Aira Visual Interpreter." Internet: <https://aira.io/>, [May 29, 2025].

Be My Eyes. "Be My Eyes , Lend your eyes to the blind." Internet: <https://www.bemyeyes.com/>, [May 29, 2025].

Microsoft. "Seeing AI." Internet: <https://www.microsoft.com/en-us/ai/seeing-ai>, [May 29, 2025].

OrCam. "OrCam MyEye 2." Internet: <https://www.orcham.com/en/myeye/>, [May 29, 2025].

Hugging Face. "Transformers Documentation." Internet: <https://huggingface.co/docs/transformers/index>, [May 29, 2025].

TensorFlow. "TensorFlow Lite." Internet: <https://www.tensorflow.org/lite>, [May 29, 2025].

Google. "ML Kit." Internet: <https://developers.google.com/ml-kit>, [May 29, 2025].

OpenCV. "Open Source Computer Vision Library." Internet: <https://opencv.org/>, [May 29, 2025].

spaCy. "Industrial-strength NLP." Internet: <https://spacy.io/>, [May 29, 2025].

Stable-Baselines3. "Reliable reinforcement learning implementations." Internet: <https://stable-baselines3.readthedocs.io/>, [May 29, 2025].

COCO Dataset. "Common Objects in Context." Internet: <https://cocodataset.org/>, [May 29, 2025].

VGGFace2. "Deep Face Recognition Dataset." Internet: [https://www.robots.ox.ac.uk/~vgg/data/vgg\\_face2/](https://www.robots.ox.ac.uk/~vgg/data/vgg_face2/), [May 29, 2025].

LibriSpeech. "ASR Corpus." Internet: <https://www.openslr.org/12>, [May 29, 2025].

Firebase. "Firebase Documentation." Internet: <https://firebase.google.com/docs>, [May 29, 2025].

Flutter. "Flutter Documentation." Internet: <https://docs.flutter.dev/>, [May 29, 2025].

Deepgram. "Nova-2 STT API Documentation." Internet: <https://developers.deepgram.com/docs/nova-2>, [May 29, 2025].

Open Food Facts. "Open Food Facts API." Internet: <https://world.openfoodfacts.org/data>, [May 29, 2025].

Twilio. "Programmable SMS and Voice API." Internet: <https://www.twilio.com/docs>, [May 29, 2025].

## 8 Chapter 8: Plagiarism Report



Page 2 of 36 - Integrity Overview

Submission ID trn:oid::3618:117182344

### 9% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

#### Filtered from the Report

- Bibliography
- Quoted Text

#### Match Groups

-  **62 Not Cited or Quoted 8%**  
Matches with neither in-text citation nor quotation marks
-  **2 Missing Quotations 0%**  
Matches that are still very similar to source material
-  **0 Missing Citation 0%**  
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**  
Matches with in-text citation present, but no quotation marks

#### Top Sources

- 4%  Internet sources
- 0%  Publications
- 7%  Submitted works (Student Papers)

#### Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.



## Appendix D

### General Coding Standards & Guidelines

1. **Naming Conventions:** camelCase for variables and functions (e.g., checkExpiryStatus, voiceCommandHandler); PascalCase for React components and TypeScript classes (e.g., PantryScreen, EmergencyManager); SCREAMING\_SNAKE\_CASE for constants (e.g., CONFIDENCE\_THRESHOLD, HIGH\_IMPACT\_MAGNITUDE).
2. **File Organization:** One service or component per file; file name matches the exported class name (e.g., pantryService.ts, EmergencyScreen.tsx). Services grouped by functional domain in src/services/ subdirectories.
3. **TypeScript Strict Mode:** All .ts and .tsx files use strict: true in tsconfig.json. No any types permitted without explicit justification. All service interfaces defined with explicit TypeScript types.
4. **Error Handling:** All AI API calls wrapped in try-catch blocks with three-tier fallback logic. Firebase operations use error boundary patterns. Failed operations are logged to Firebase Analytics with error\_type and module\_id tags.
5. **Comments and Documentation:** JSDoc comments on all exported functions and service classes. Roman Urdu audio strings documented in comments adjacent to Speech.speak() calls.
6. **DRY Principle:** Shared utilities in src/utils/; shared TypeScript types in src/models/; no business logic duplicated across services.
7. **Single Responsibility:** Each service handles exactly one domain (e.g., productRecognition.service.ts handles only YOLOv8 detection; pantryStorage.service.ts handles only Firestore pantry CRUD).
8. **Accessibility Code Standards:** All interactive React Native components include accessibilityLabel and accessibilityHint props. All TextInput fields include accessibilityRole='button' or appropriate role. Minimum touch target size of 48×48dp enforced via StyleSheet minWidth/minHeight.